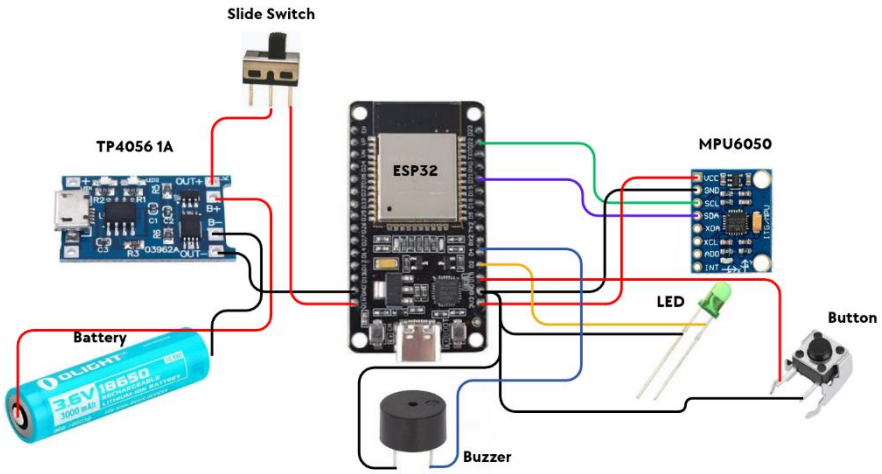
บทที่ 3
วิธีดำเนินงาน

การดำเนินโครงงานและพัฒนาระบบตรวจจับการหกล้มอัจฉริยะสำหรับผู้สูงอายุด้วยปัญญาประดิษฐ์ฝังตัวบนไมโครคอนโทรลเลอร์ ได้รับการออกแบบตามวงจรชีวิตการพัฒนาระบบวิศวกรรมแบบครบวงจร (Complete Engineering Lifecycle) ตั้งแต่การสร้างฮาร์ดแวร์และกล่องเคส 3 มิติ การสร้างเว็บแอปพลิเคชันเพื่อเก็บรวบรวมข้อมูลสัญญาณดิบด้วยตนเอง การฝึกสอนโมเดลปัญญาประดิษฐ์ การฝังโมเดลลงบนฮาร์ดแวร์ ตลอดจนการพัฒนาเว็บแอปพลิเคชันสำหรับผู้ดูแลเพื่อนำไปใช้งานจริง โดยแบ่งขั้นตอนการดำเนินงานออกเป็น 6 หัวข้อหลักดังต่อไปนี้

3.1 เตรียมอุปกรณ์ฮาร์ดแวร์ สำหรับเก็บข้อมูลอาสาสมัคร

การออกแบบฮาร์ดแวร์ สำหรับเก็บข้อมูลอาสาสมัคร ในการเก็บรวบรวมข้อมูลสัญญาณอนุกรมเวลาจากการเคลื่อนไหวของร่างกายเพื่อนำมาใช้ฝึกสอนและประเมินผลแบบจำลองปัญญาประดิษฐ์ฝังตัว การใช้อุปกรณ์บันทึกข้อมูลทั่วไปหรือการถือสมาร์ตโฟนในมืออาจก่อให้เกิดความคลาดเคลื่อนทางจลนศาสตร์สูง เนื่องจากเซ็นเซอร์ไม่ได้ยึดติดแน่นกับจุดศูนย์ถ่วงของร่างกายและเกิดการเคลื่อนที่สัมพัทธ์ (Relative Motion) ผู้วิจัยจึงได้ออกแบบและสร้างอุปกรณ์ฮาร์ดแวร์ต้นแบบเฉพาะทางสำหรับเก็บข้อมูลอาสาสมัครขึ้น (CollectData_Hardware) โดยมีวัตถุประสงค์เพื่อทำหน้าที่เป็นอุปกรณ์บันทึกข้อมูลความเคลื่อนไหวเชิงเฉื่อยแบบสวมใส่ (Wearable Inertial Data Logger) ที่มีความเที่ยงตรงทางเวลาสูง น้ำหนักเบา ทนทานต่อแรงกระแทก และสามารถส่งผ่านข้อมูลไปยังระบบหลังบ้านได้อย่างแม่นยำ โดยมีรายละเอียดการออกแบบเชิงวิศวกรรมในแต่ละส่วนดังนี้

ด้านการออกแบบวงจร CollectData_Hardware



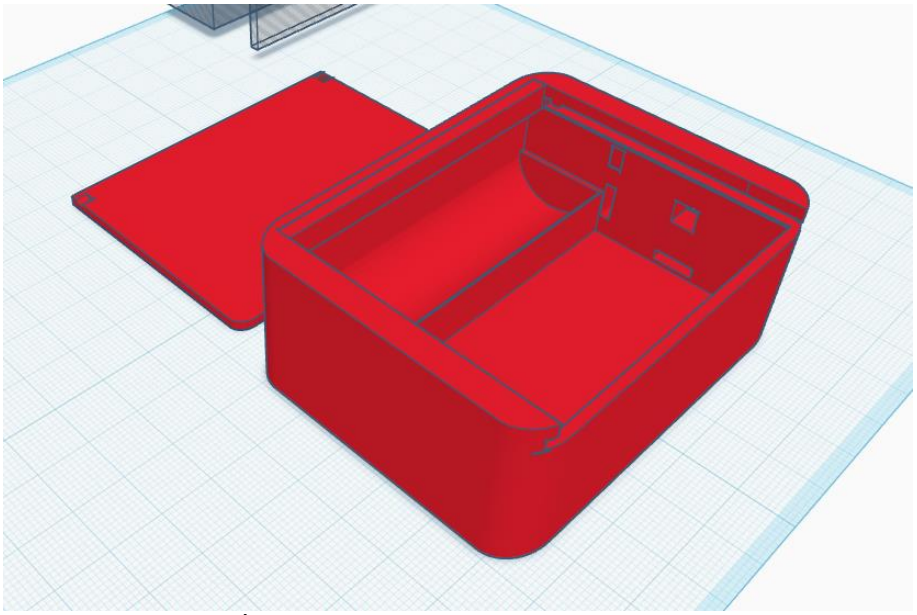
รูปที่ 3.1 (ก) วงจรCollectData_Hardware เบื้องต้น

รายละเอียดการเชื่อมต่อขาสัญญาณของอุปกรณ์ฮาร์ดแวร์

ตารางที่ 3.1.1.2 รายละเอียดการเชื่อมต่อขาสัญญาณของอุปกรณ์ฮาร์ดแวร์

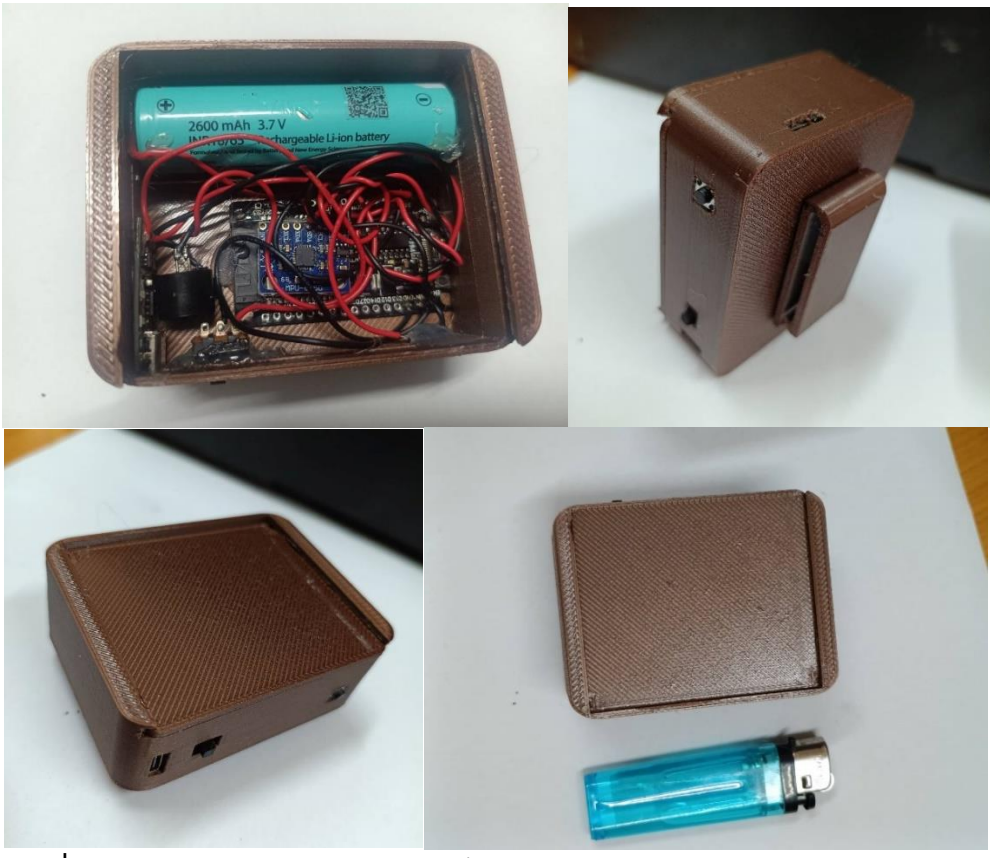
โมดูล / อุปกรณ์	**ขาสัญญาณบนโมดูล**	**ขาสัญญาณบน ESP32**	**โพรโทคอล / ฟังก์ชันการทำงาน**
MPU-6050 (IMU)	VCC	3.3V	แหล่งจ่ายไฟกระแสตรงแรงดันคงที่
	GND	GND	กราวด์ร่วมของระบบ
	SDA	GPIO 21	I^2CData (Fast Mode 400 kHz)
	SCL	GPIO 22	I^2CClock (Fast Mode 400 kHz)
Buzzer	I/O (+)	GPIO 5	สัญญาณพัลส์ขับเสียงบี๊บนำทาง (2.4 kHz)
	GND (-)	GND	กราวด์ร่วม
Button	ขา 1	GPIO 18	สวิตช์กดติด-ปล่อยดับ กดเริ่มบันทึกท่าADL
	ขา 2	GND	ใช้วงจร Internal Pull-Up Resistor
Status LED	ขั้ว Anode	GPIO 2	ไฟแสดงสถานะการทำงาน (Onboard LED)
Battery Unit	ขั้วบวก/ลบ	5V / GND	แบตเตอรี่ Li-ion 3.7V ผ่านวงจร TP 4056 1A Boost 5V

สร้างโมเดล 3 มิติ เพื่อให้ตัวอุปกรณ์สามารถสวมใส่ได้สบาย มีความทนทานต่อแรงกระแทก และล็อกตำแหน่งเซ็นเซอร์ MPU-6050 ได้อย่างมั่นคง ผู้จัดทำโครงงานได้ออกแบบโครงสร้างกล่องเคสแบบ 2 ชิ้นประกบ



รูปที่ 3.1(ข) โมเดล 3 มิติของกล่องเคสอุปกรณ์

รวมอุปกรณ์และเคสโมเดล3มิติ



รูปที่ 3.1(ค) อุปกรณ์ฮาร์ดแวร์ สำหรับเก็บข้อมูลอาสาสมัคร CollectData_Hardware

โค้ตทำงานด้านการเก็บข้อมูลอาสาสมัคร เนื่องด้วยการที่จะเทรมโมเดลAI สำหรับการล้ม จึงต้องพัฒนาโค้ตด้านเก็บข้อมูลสัญญาณ 6 แกน($A_x, A_y, A_z, \omega_x, \omega_y, \omega_z$)ให้พร้อมกับการเป็นท่าทางหลัก 2 ประเภท คือ

1) ท่าล้ม ไมโครคอนโทรลเลอร์จะทำการเป็นค่าเมื่อเกิดการล้มในท่าที่กำหนด ก่อนและหลังล้ม ภายใน 2 วินาที (50Hz)

```
// บันทึกรวมข้อมูล 50 จุดก่อนล้ม + 50 จุดหลังล้ม ลงใน 1 แพ็กเก็ตเดียว (100 จุด = 1 HTTP POST)
activePacket = getFreePacket();
if (activePacket) {
  activePacket->record_id = activeRecordId;
  activePacket->batch_index = 0;
  activePacket->sample_count = 0;
  activePacket->is_final = false;

  // ดึงข้อมูล 50 จุด (1.0 วินาที) ก่อนเกิดแรงกระแทกจาก Circular Buffer
  // รับประกันว่า activePacket จะได้ข้อมูลก่อนล้มครบ 50 จุดแน่นอน 100%
  int availablePre = (preCount > PRE_EVENT_SAMPLES) ? PRE_EVENT_SAMPLES : preCount;
  int padCount = PRE_EVENT_SAMPLES - availablePre;

  // กรณีมีข้อมูลก่อนล้มไม่ถึง 50 จุด ให้เติมตัวอย่างแรกสุดซ้ำเพื่อรับประกันว่าครบ 50 จุดเสมอ
  if (padCount > 0) {
    int oldestIdx = (preCount > 0) ? ((preWriteIndex - preCount + PRE_EVENT_SAMPLES) % PRE_EVENT_SAMPLES) : 0;
    SensorSample padSample = (preCount > 0) ? preBuffer[oldestIdx] : s;
    for (int i = 0; i < padCount; i++) {
      SensorSample ps = padSample;
      ps.timestamp_ms = i * SAMPLE_RATE_MS;
      activePacket->samples[activePacket->sample_count++] = ps;
    }
  }

  int startIdx = (preWriteIndex - availablePre + PRE_EVENT_SAMPLES) % PRE_EVENT_SAMPLES;
  for (int i = 0; i < availablePre; i++) {
    SensorSample ps = preBuffer[(startIdx + i) % PRE_EVENT_SAMPLES];
    ps.timestamp_ms = (padCount + i) * SAMPLE_RATE_MS; // 0, 20, 40, ... 980 ms
    activePacket->samples[activePacket->sample_count++] = ps;
  }
}

postFallSampleCount = 0;
currentMode = MODE_RECORDING_POST_FALL;
}

// หมดเวลา Timeout 60 วินาที หากไม่มีการล้มเกิดขึ้น
if (now - waitFallStartTime > 60000 || stopContinuousRequested) {
  Serial.println("[Core 1] ⌛ หมดเวลารอตรวจจับการล้ม หรือผู้ใช้สั่งยกเลิก");
  currentMode = MODE_FINISHING;
```

รูปที่ 3.1(ง) ฟังก์ชั่นเก็บข้อมูลท่าล้มของ CollectData_Hardware (C/C++)

2) ท่าADL(กิจกรรมปกติ) ไมโครคอนโทรลเลอร์จะทำการเป็นค่าเมื่อถูกสั่งการ ภายใน 10 วินาที (50Hz) โดยเริ่มจาก วินาทีที่ 1 เตรียมพร้อมด้วยท่ายืน, วินาทีที่ 2-9 ทำท่าทางที่จะเก็บบันทึก 1 ครั้ง และจบวินาทีที่ 10 ด้วยการกลับมาท่ายืนดังเดิม



```
if (activePacket) {
  activePacket->samples[activePacket->sample_count++] = s;
  adlSampleCount++;

  // สัญญาณเสียงขั้นตอนที่ 1: ครบ 1 วินาที (ตัวอย่างที่ 50) -> ลำโพงดัง Beep! ให้เริ่มท่าท่า
  if (adlSampleCount == 50) {
    startBeepAsync(150);
    Serial.println("  [Core 1] 📢 ลำโพงดัง Beep! -> สเต็ปที่ 2: เริ่มท่าท่าทางที่กำหนด (8 วินาที)...");
  }

  // สัญญาณเสียงขั้นตอนที่ 2: ครบ 9 วินาที (ตัวอย่างที่ 450) -> ลำโพงดัง Beep! ให้กลับมาท่ายืน
  if (adlSampleCount == 450) {
    startBeepAsync(150);
    Serial.println("  [Core 1] 📢 ลำโพงดัง Beep! -> สเต็ปที่ 3: กลับมาท่ายืนนิ่งๆ (1 วินาที)...");
  }

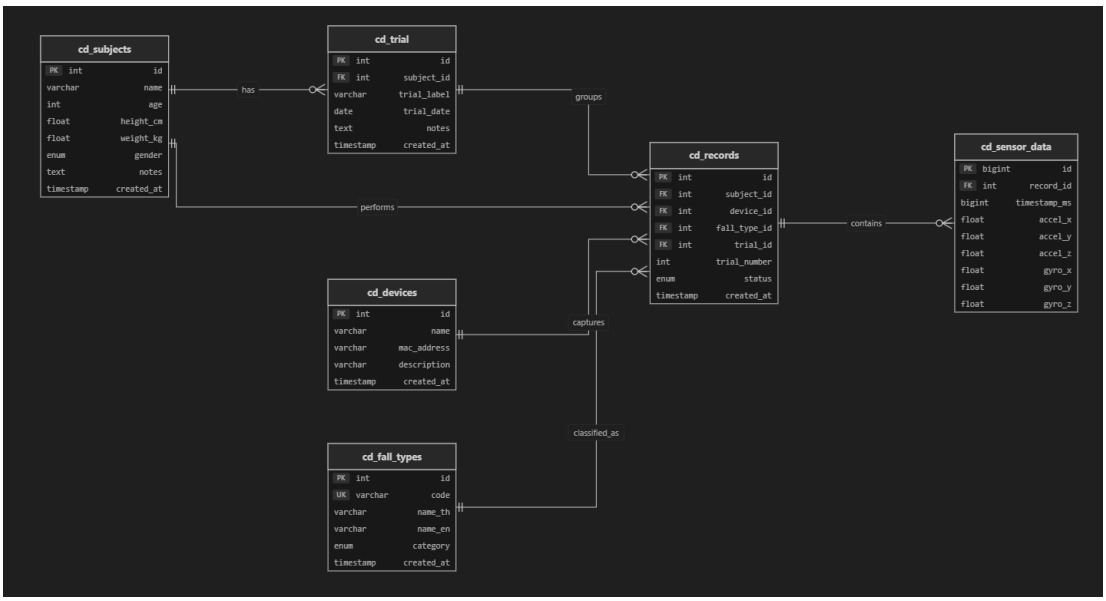
  // เมื่อเก็บครบ 500 จุด (10 วินาที) -> ไปโหมดส่งเสียงจบก่อนแล้วค่อยส่งขึ้น Core 0
  if (activePacket->sample_count >= BATCH_SIZE) {
    bool isLastBatch = (adlSampleCount >= 500) || stopContinuousRequested;
    if (isLastBatch) {
      activePacket->is_final = true;
      Serial.println("[Core 1] ✓ บันทึกรอบ ADL ครบถ้วน 500 จุดข้อมูล (10.0 วินาที)");
      currentMode = MODE_FINISHING;
    } else {
      if (xQueueSend(batchUploadQueue, &activePacket, 0) == pdTRUE) {
        activePacket = NULL;
        currentBatchIndex++;
      } else {
        spoolBatchToSpiffs(activePacket);
        releasePacket(activePacket);
        activePacket = NULL;
        currentBatchIndex++;
      }
    }
  }
}

if (stopContinuousRequested && currentMode != MODE_FINISHING) {
  currentMode = MODE_FINISHING;
```

รูปที่ 3.1(จ) ฟังก์ชั่นเก็บข้อมูลท่าADLของ CollectData_Hardware (C/C++)

3.2 สร้างเว็บแอพสำหรับกระบวนการเก็บข้อมูลอาสาสมัคร

การฝึกสอนแบบจำลองปัญญาประดิษฐ์ให้มีความแม่นยำและสามารถตรวจจับการหกล้มได้อย่างมีเสถียรภาพ จำเป็นต้องอาศัยชุดข้อมูลสัญญาณดิบ (Raw Time-Series Data) ที่บันทึกจากการเคลื่อนไหวจริงของมนุษย์ในสภาพแวดล้อมที่ควบคุม อย่างไรก็ตาม กระบวนการเก็บข้อมูลจากการทดลองจริงมีความท้าทายหลายประการ เช่น ความเสี่ยงต่อความปลอดภัยของอาสาสมัคร ความคลาดเคลื่อนจากการกดบันทึกสัญญาณไม่ตรงจังหวะ และความไม่เป็นระเบียบของโครงสร้างข้อมูล ด้วยเหตุนี้ ผู้จัดทำโครงงานจึงได้พัฒนาระบบเว็บแอปพลิเคชันเฉพาะกิจขึ้นมาในชื่อ **CollectData_Web** เพื่อทำหน้าที่เป็นศูนย์กลางควบคุมการทดลองแบบบูรณาการ (Experimentation & Data Collection Hub) สำหรับประสานงานระหว่างผู้วิจัย อาสาสมัคร และอุปกรณ์สวมใส่ฮาร์ดแวร์ ออกแบบเว็บแอพสำหรับกระบวนการเก็บข้อมูลอาสาสมัครดังรูปภาพดังนี้



รูปที่ 3.2 การออกแบบโครงสร้างฐานข้อมูลเชิงสัมพันธ์

ตารางที่ 3.2 รายละเอียดพจนานุกรมข้อมูล (Data Dictionary) ของระบบ CollectData_Web

ชื่อตาราง	ฟิลด์ข้อมูล (Field)	ชนิดข้อมูล (Data Type)	คีย์ / ดัชนี	คำอธิบายหน้าที่
cd_subjects	id	INT	PRIMARY KEY, AUTO_INC	รหัสประจำตัวอาสาสมัคร
	name	VARCHAR(100)	NOT NULL	ชื่อ-นามสกุลของอาสาสมัคร
	age	INT	NULLABLE	อายุ (ปี)
	height_cm	FLOAT	NULLABLE	ส่วนสูง (เซนติเมตร)
	weight_kg	FLOAT	NULLABLE	น้ำหนักตัว (กิโลกรัม)
	gender	ENUM('M','F','O')	DEFAULT 'O'	เพศ (M=ชาย, F=หญิง, O=อื่น ๆ)
	notes	TEXT	NULLABLE	หมายเหตุหรือข้อจำกัดทางกายภาพ
cd_devices	id	INT	PRIMARY KEY, AUTO_INC	รหัสอุปกรณ์ในระบบ

ตารางที่ 3.2 รายละเอียดพจนานุกรมข้อมูล (Data Dictionary) ของระบบ CollectData_Web (ต่อ)

ชื่อตาราง	ฟิลด์ข้อมูล (Field)	ชนิดข้อมูล (Data Type)	คีย์ / ดัชนี	คำอธิบายหน้าที่
	name	VARCHAR(50)	NOT NULL	ชื่ออุปกรณ์
	mac_address	VARCHAR(17)	INDEX	ค่า Physical Address (Wi
	description	VARCHAR(200)	NULLABLE	รายละเอียดตำแหน่งติดตั้งบอร์ด
cd_fall_types	id	INT	PRIMARY KEY, AUTO_INC	รหัสลำดับประเภทท่าทาง
	code	VARCHAR(20)	UNIQUE, NOT NULL	รหัสอ้างอิงมาตรฐาน (F01–F05, D01–D06)
	name_th	VARCHAR(100)	NOT NULL	ชื่อท่าทางภาษาไทย
	name_en	VARCHAR(100)	NULLABLE	ชื่อท่าทางภาษาอังกฤษ
	category	ENUM	'fall', 'adl', 'other'	การจัดกลุ่มหมวดหมู่หลัก
cd_records	id	INT	PRIMARY KEY, AUTO_INC	รหัสรอบการบันทึกสัญญาณ
	subject_id	INT	FOREIGN KEY	เชื่อมโยงไปยังอาสาสมัครที่ทดสอบ
	device_id	INT	FOREIGN KEY	เชื่อมโยงไปยังอุปกรณ์ที่ใช้บันทึก
	fall_type_id	INT	FOREIGN KEY	เชื่อมโยงไปยังประเภทท่าทางที่ทดสอบ

ตารางที่ 3.2 รายละเอียดพจนานุกรมข้อมูล (Data Dictionary) ของระบบ CollectData_Web (ต่อ)

ชื่อตาราง	**ฟิลด์ข้อมูล (Field)**	**ชนิดข้อมูล (Data Type)**	**คีย์ / ดัชนี**	**คำอธิบายหน้าที่**
	trial_number	INT	DEFAULT 1	ลำดับการทดสอบซ้ำในท่านั้น (1–5)
	status	ENUM	'recording', 'completed', 'cancelled'	สถานะวงรอบการบันทึกข้อมูล
cd_sensor_data	id	BIGINT	PRIMARY KEY, AUTO_INC	รหัสจุดข้อมูลสัญญาณ
	record_id	INT	FOREIGN KEY (CASCADE)	เชื่อมโยงไปยังรอบการบันทึก (cd_records)
	timestamp_ms	BIGINT	NOT NULL	เวลาแสตมป์จากตัวนับของบอร์ด (ms)
	accel_x, y, z	FLOAT	NOT NULL	ค่าความเร่งเชิงเส้น 3 แกน (m/s^2)
	gyro_x, y, z	FLOAT	NOT NULL	ค่าความเร็วเชิงมุม 3 แกน
	INDEX	Compound	idx_record_time (record_id, timestamp_ms)	ดัชนีเพิ่มความเร็วในการดึงข้อมูลอนุกรมเวลา

3.3 การกำหนดออนโทโลจีท่าทางมาตรฐาน

เพื่อความเป็นระเบียบและสอดคล้องกับระเบียบวิธีวิจัยสากล ระบบได้กำหนดประเภทท่าทางการทดลองมาตรฐานไว้ล่วงหน้า 11 ท่าทาง ประกอบด้วยการล้ม 5 ท่าทาง และกิจกรรมประจำวัน 6 ท่าทาง โดยกำหนดกรอบเวลาการบันทึกที่ชัดเจน

3.3.1 หมวดการล้ม (Fall Category: F01–F05) — ระยะเวลาบันทึก 2.0 วินาที

1) F01 (Fall Forward) การล้มไปข้างหน้าจากการสะดุดหรือเสียการทรงตัว

2) F02 (Fall Backward) การล้มไปข้างหลังจากการลื่นไถล

3) F03 (Fall Left) การล้มตะแคงไปทางด้านซ้าย

4) F04 (Fall Right) การล้มตะแคงไปทางด้านขวา

5) F05 (Vertical Fall) การล้มทรุดตัวลงในแนวดิ่ง (เช่น อาการเข่าอ่อนหรือหมดสติฉับพลัน)

ช่วงเวลา 2.0 วินาที ครอบคลุมพฤติกรรมช่วงก่อนล้ม 1.0 วินาทีและช่วงหลังการกระแทก 1.0 วินาที อย่างสมบูรณ์

3.3.2 หมวดกิจกรรมปกติในชีวิตประจำวัน (ADL Category: D01–D06) — ระยะเวลาบันทึก10.0 วินาที

1) D01 (Walking) การเดินต่อเนื่องในทางราบ

2) D02 (Jogging) การวิ่งเหยาะหรือกึ่งวิ่งกึ่งเดิน

3) D03 (Upstairs) การเดินก้าวขึ้นบันได

4) D04 (Bending) การก้มตัวลงเก็บสิ่งของที่พื้นแล้วยืนขึ้น

5) D05 (Sit to Stand) การนั่งลงบนเก้าอี้ปกติแล้วลุกขึ้นยืน

6) D06 (Quick Sit to Stand) การทิ้งตัวนั่งลงอย่างรวดเร็วและลุกขึ้นยืนฉับพลัน

ช่วงเวลา10.0 วินาที มีลำดับขั้นการทดลองคือ ยืนนิ่งเตรียมตัว 1.0 วินาที แรก ปฏิบัติกิจกรรม 8.0 วินาที และกลับมายืนนิ่งใน1.0 วินาที สุดท้าย

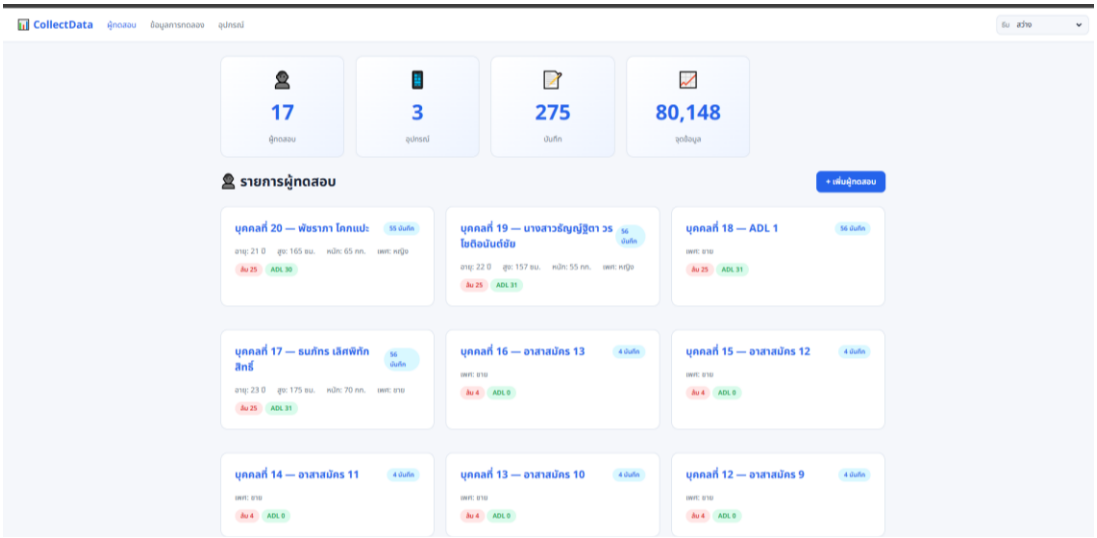
3.3.3 การออกแบบส่วนต่อประสานผู้ใช้และคอนโซลควบคุมการทดลอง (UI/UX Console Design)

3.3.3.1 หน้าแดชบอร์ดภาพรวมระบบ (index.php)

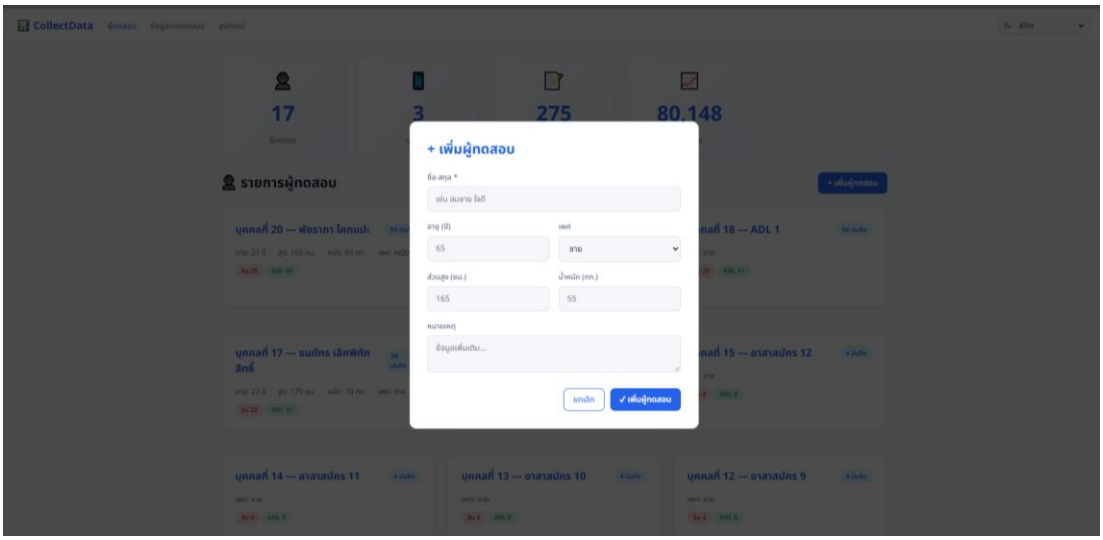
• ทำหน้าที่เป็นศูนย์กลางการติดตามความคืบหน้าของโครงการ มีการ์ดสรุปตัวเลขสถิติแบบเรียลไทม์ (KPI Summary Cards) ได้แก่ จำนวนอาสาสมัครทั้งหมด, จำนวนอุปกรณ์ฮาร์ดแวร์ที่ลงทะเบียน, จำนวนรอบการทดลองที่บันทึกแล้วเสร็จ, และยอดรวมจุดข้อมูลสัญญาณเซ็นเซอร์ทั้งหมดในระบบ

• แสดงตารางรายชื่ออาสาสมัครพร้อมปุ่มการกระทำด่วน (Quick Actions) ได้แก่ การเปิดหน้าคอนโซลทดสอบรายบุคคล (subject.php), การแก้ไขข้อมูลกายภาพ, และการลบข้อมูล

• มีฟอร์ม Modal สำหรับเพิ่มอาสาสมัครใหม่ โดยระบบจะตรวจสอบความถูกต้องของข้อมูล (Data Validation) เช่น ส่วนสูงและน้ำหนักต้องมีค่ามากกว่าศูนย์



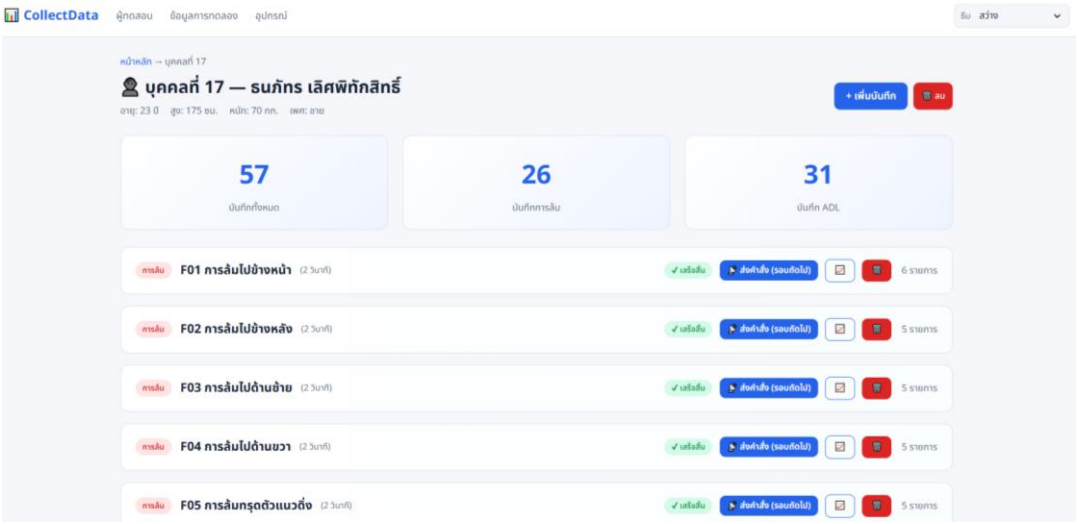
รูปที่ 3.3.3.1(ก) หน้าแดชบอร์ดภาพรวมระบบ CollectData_Web



รูปที่ 3.3.3.1(ข) ฟอร์ม Modal สำหรับเพิ่มอาสาสมัครใหม่ CollectData_Web

3.3.3.2 คอนโซลควบคุมการทดสอบอาสาสมัคร

ก) หน้าจอหลักที่ผู้วิจัยใช้งานระหว่างการจำลองเหตุการณ์จริง แสดงรายละเอียดประวัติของอาสาสมัครที่เลือกและช่องเลือกอุปกรณ์ ESP32 ที่กำลังใช้งาน



รูปที่ 3.3.3.2 (ก) หน้าจอหลักที่ผู้วิจัยใช้งานระหว่างการจำลองเหตุการณ์จริง

ข) กลไกการสั่งการระยะไกล (Remote Triggering) มีปุ่ม "เริ่มรอบถัดไปอัตโนมัติ" (Start Next Trial) เมื่อผู้วิจัยกดปุ่ม ระบบจะค้นหารอบที่ยังว่างอยู่ และส่งคำสั่ง recording ไปรอไว้ในตาราง cd_records ทันที พร้อมระบบนับถอยหลังบนหน้าจอ



รูปที่ 3.3.3.2 (ข) ตำแหน่งปุ่ม "เริ่มรอบถัดไปอัตโนมัติ"

ค) มีหน้าต่างแสดงข้อมูลที่เก็บของท่าในแต่ละรอบออกมาเป็นกราฟเส้น เพื่อให้ง่ายต่อการดูผลการเก็บข้อมูลจากอาสาสมัคร



รูปที่ 3.3.3.2 (ค) กราฟเชิงเส้นแสดงข้อมูลที่เก็บได้ในท่า

ง) ระบบตรวจสอบสถานะแบบไม่ประสานเวลา (Asynchronous Status Polling): หน้าเว็บจะรันฟังก์ชัน JavaScript ตรวจสอบสถานะไปยัง API ทุก ๆ 1.5 วินาที เมื่อบอร์ด ESP32 อัปโหลดข้อมูลเสร็จสิ้น หน้าจอจะอัปเดตป้ายสถานะเป็นสีเขียวและเปิดให้คลิกตรวจสอบกราฟได้ทันทีโดยไม่ต้องรีเฟรชหน้าเว็บ

3.4 เก็บข้อมูล Fall และ ADL

กระบวนการเก็บข้อมูลแต่ละท่าทางที่กำหนด 2 ประเภทหลักได้แก่

3.4.1 ท่าทางการล้ม(F01-F05)ดังนี้

3.4.1.1 F01 การล้มไปข้างหน้า



รูปที่ 3.4.1.1 (ก) ตัวอย่างการล้มไปข้างหน้า



รูปที่ 3.4.1.1 (ข) กราฟตัวอย่างการล้มไปข้างหน้าที่ได้

3.4.1.2 F02 การล้มไปข้างหลัง

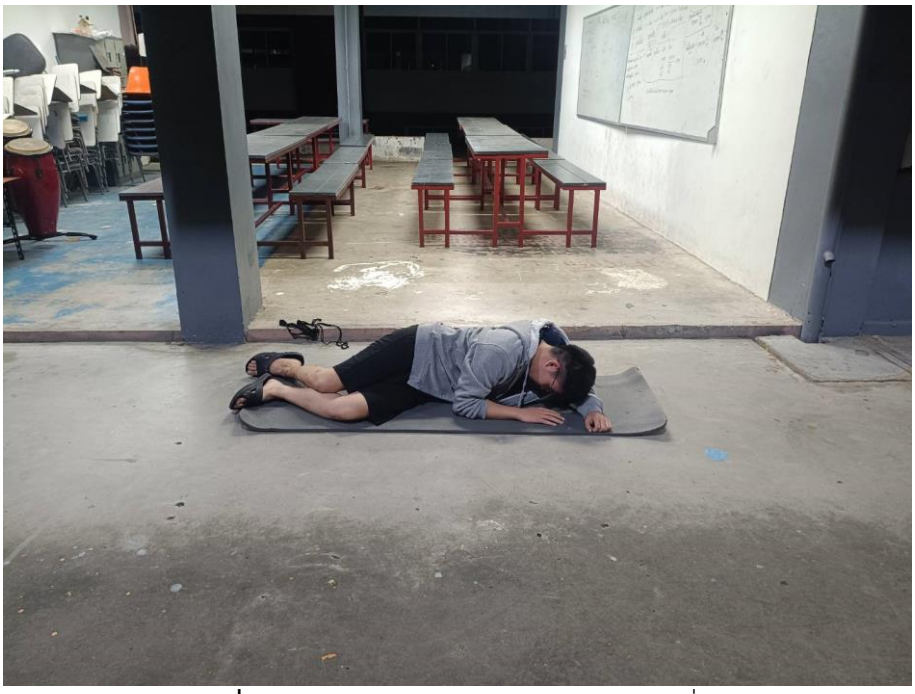


รูปที่ 3.4.1.2 (ก) ตัวอย่างการล้มไปข้างหลังที่ได้



รูปที่ 3.4.1.2 (ข) กราฟตัวอย่างการล้มไปข้างหลังที่ได้

3.4.1.3 F03 การล้มไปข้างซ้าย



รูปที่ 3.4.1.3 (ก) ตัวอย่างการล้มไปข้างซ้ายที่ได้

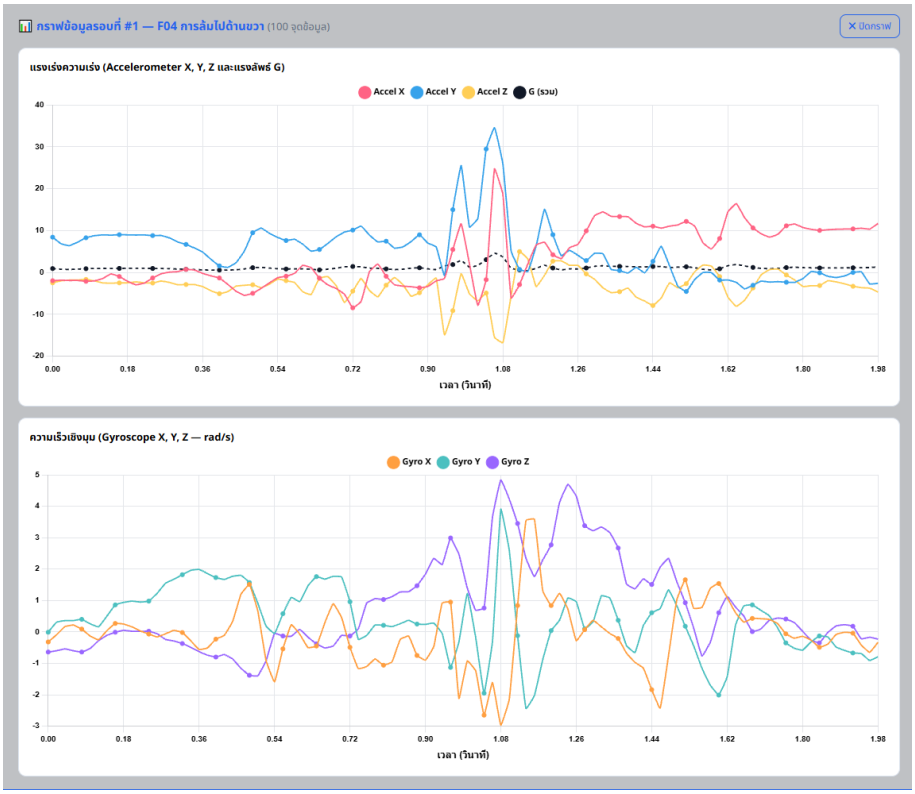


รูปที่ 3.4.1.3 (ข) กราฟตัวอย่างการล้มไปข้างซ้ายที่ได้

3.4.1.4 F04 การล้มไปข้างขวา



รูปที่ 3.4.1.4 (ก) ตัวอย่างการล้มไปข้างขวาที่ได้



รูปที่ 3.4.1.4(ข) กราฟตัวอย่างการล้มไปข้างขวาที่ได้

3.4.1.5 F05 การล้มทรุดตัวแนวดิ่ง



รูปที่ 3.4.1.5 (ก) ตัวอย่างการล้มทรุดตัวแนวดิ่งที่ได้



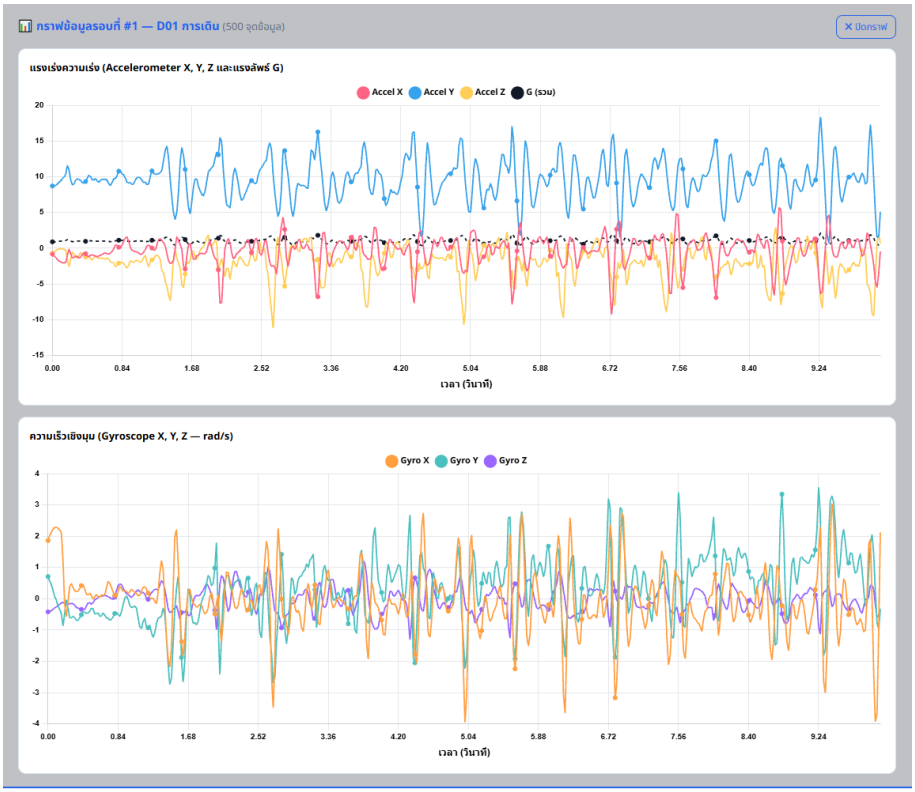
รูปที่ 3.4.1.5 (ข) กราฟตัวอย่างการล้มทรุดตัวแนวดิ่งที่ได้

3.4.2 ท่าทางกิจกรรมท่าไป(D01-D06)ดังนี้

3.4.2.1 D01 การเดิน

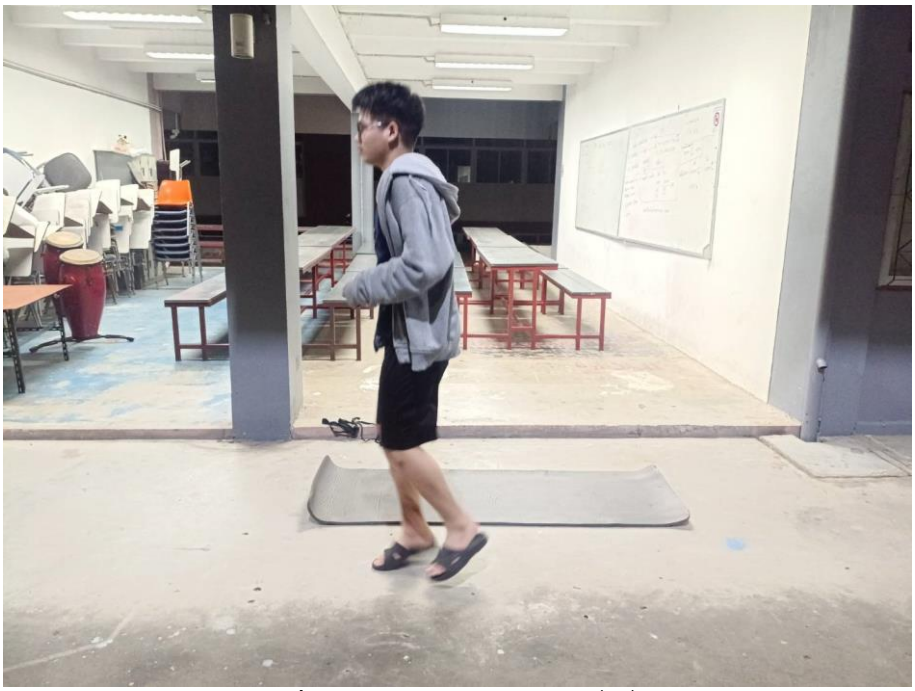


รูปที่ 3.4.2.1(ก) ตัวอย่างการเดินที่ได้

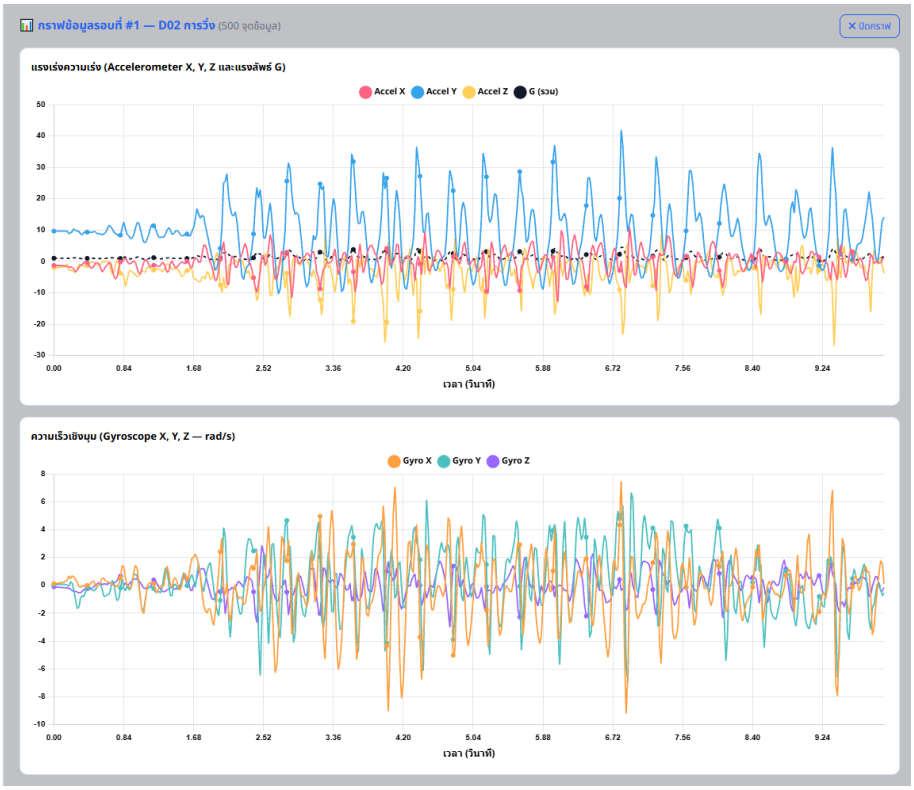


รูปที่ 3.4.2.1 (ข) กราฟตัวอย่างการเดินที่ได้

3.4.2.2 D02 การวิ่ง



รูปที่ 3.4.2.2 (ก) ตัวอย่างการวิ่งที่ได้

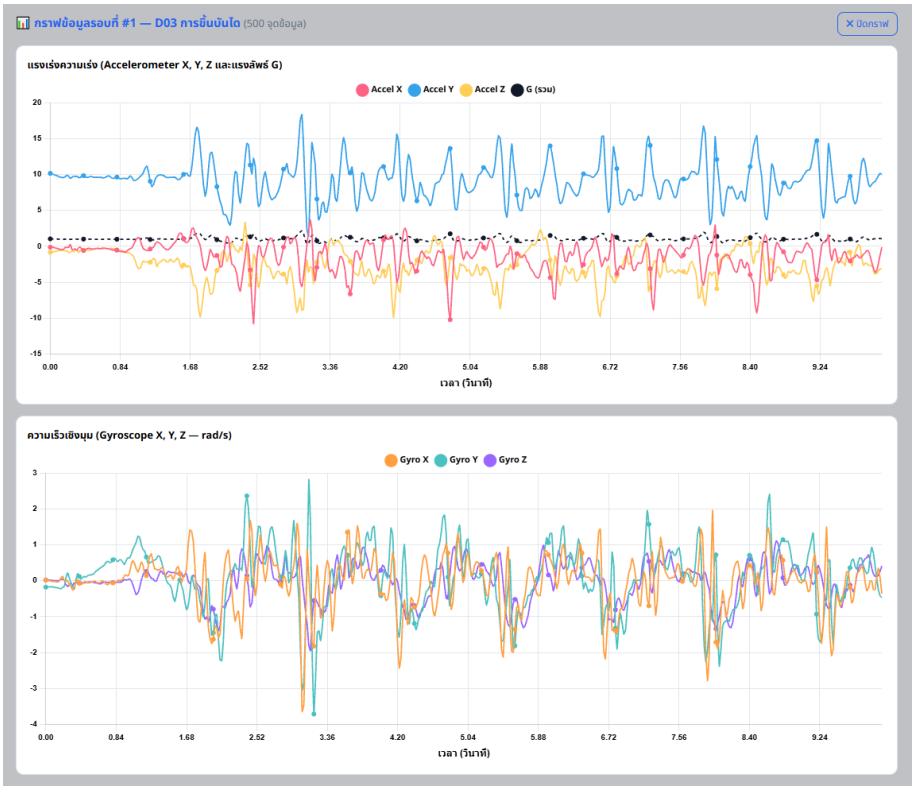


รูปที่ 3.4.2.2 (ข) กราฟตัวอย่างการวิ่งที่ได้

3.4.2.3 D03 การเดินบันได

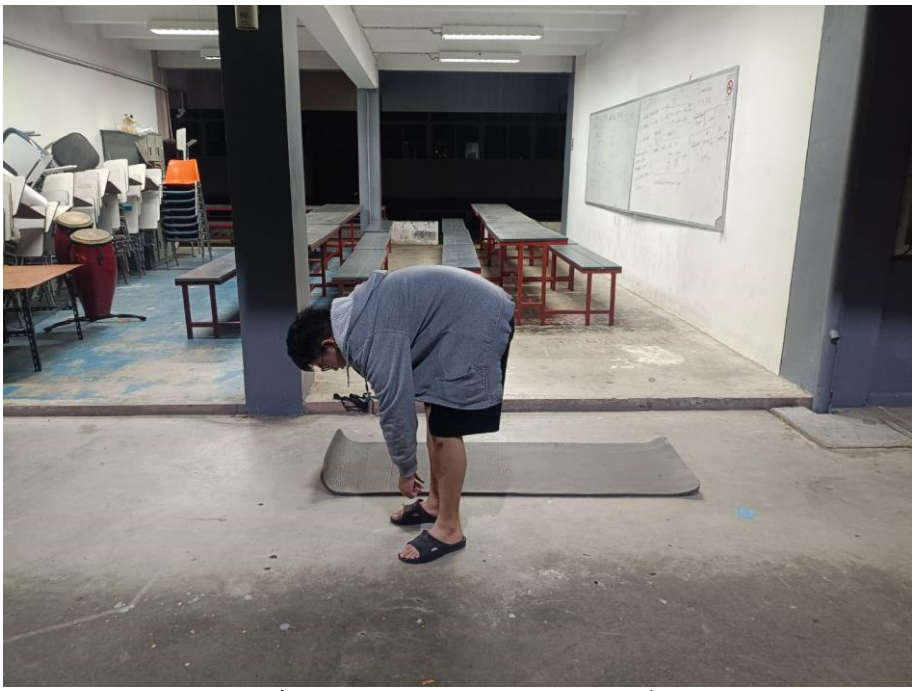


รูปที่ 3.4.2.3 (ก) ตัวอย่างการเดินบันไดที่ได้

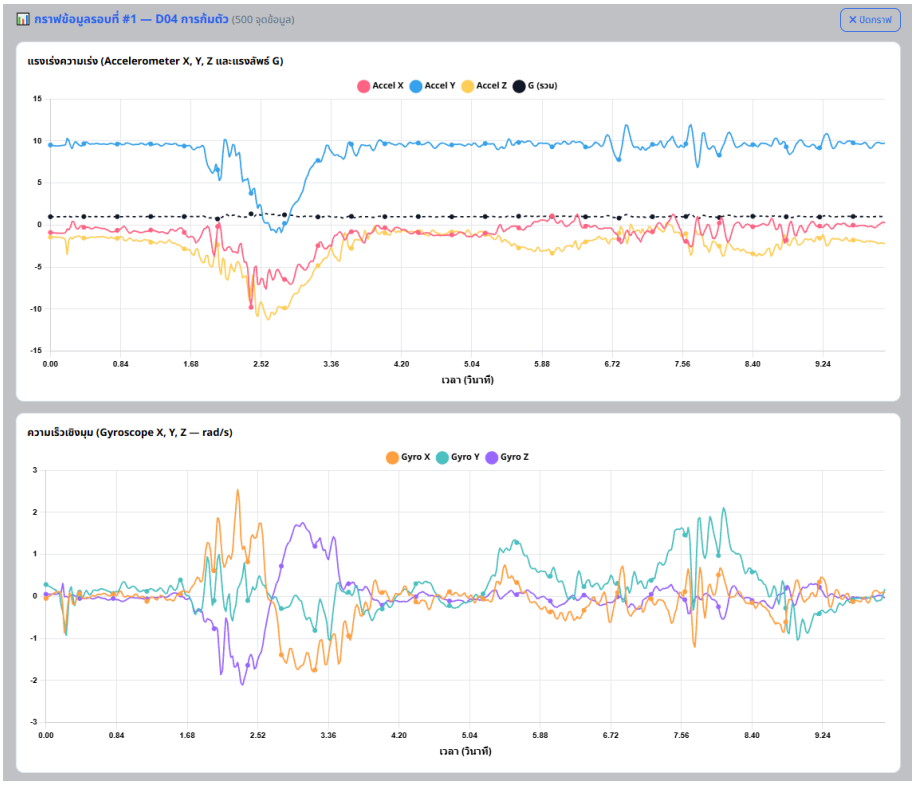


รูปที่ 3.4.2.3 (ข) กราฟตัวอย่างการเดินบันไดที่ได้

3.4.2.4 D04 การก้มตัว



รูปที่ 3.4.2.4 (ก) ตัวอย่างการก้มตัวที่ได้

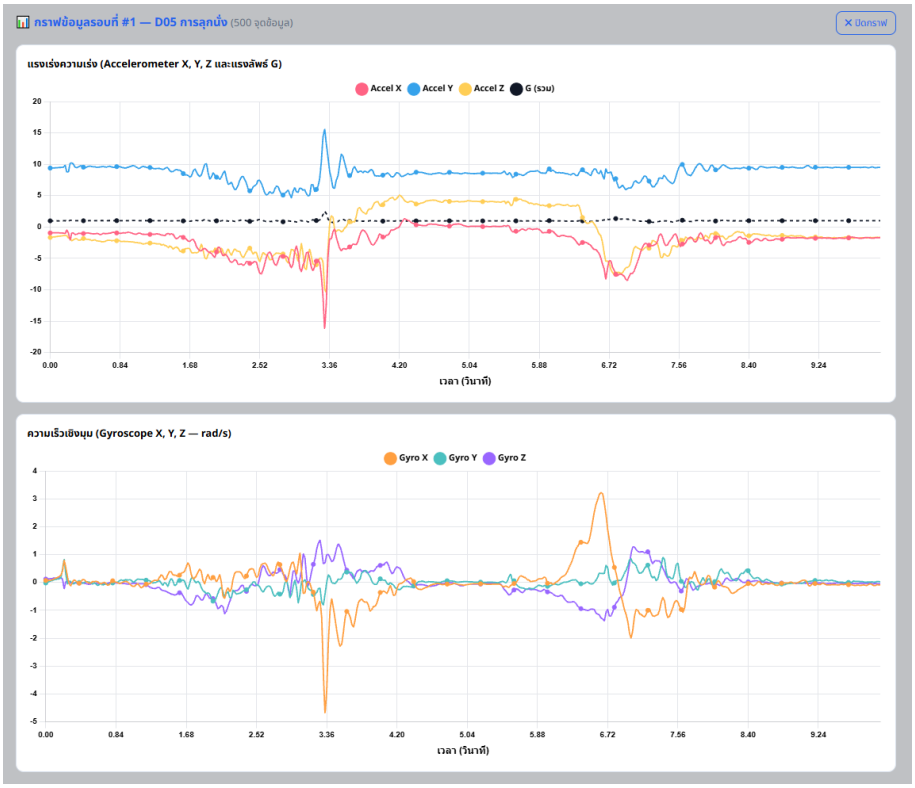


รูปที่ 3.4.2.4 (ข) กราฟตัวอย่างการก้มตัวที่ได้

3.4.2.5 D05 การลุกนั่ง



รูปที่ 3.4.2.5 (ก) ตัวอย่างการลุกนั่งที่ได้

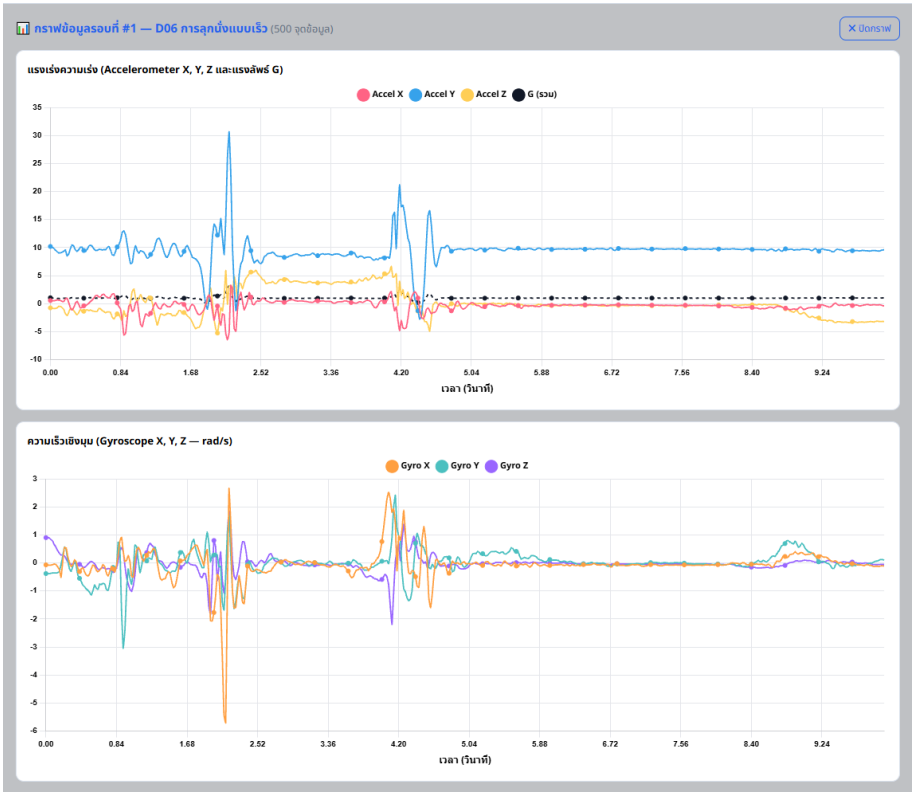


รูปที่ 3.4.2.5 (ข) กราฟตัวอย่างการลุกนั่งที่ได้

3.4.2.6 D06 การลุกนั่งแบบเร็ว



รูปที่ 3.4.2.6 (ก) ตัวอย่างการลุกนั่งแบบเร็วที่ได้



รูปที่ 3.4.2.6 (ข) กราฟตัวอย่างการลุกนั่งแบบเร็วที่ได้

3.5 นำข้อมูลที่ได้จากมาสร้างโมเดลAI

แหล่งข้อมูล CollectData_Web (ESP32 + MPU-6050 ติดตั้งบริเวณกึ่งกลางเอว)

จำนวนตัวอย่างทั้งหมด 222 (Fall 100 รอบ + ADL 122 รอบ)

3.5.1 แบ่งแบบ 80 : 20 (Stratified Train-Test Split)

เป็นวิธีมาตรฐานที่ใช้ในการประเมินผลแบบจำลอง (Holdout Method) โดยแบ่งข้อมูลออกเป็น 2 ส่วน

เนื่องจากข้อมูลประกอบด้วย 2 คลาส คือ **ท่าล้ม (Fall)** และ **ท่าปกติ (ADL)** หากสุ่มแบบธรรมดา อาจทำให้ชุดทดสอบมีท่าล้มมากหรือน้อยเกินไป การใช้ Stratified จะรักษาสัดส่วนคลาสให้เท่ากันทั้ง 2 ชุด

- **Train Set (80%):** 178 ตัวอย่าง (Fall = 80, ADL = 98)
- **Test Set (20%):** 45 ตัวอย่าง (Fall = 20, ADL = 25)

```python
def apply_moving_average_6axis(raw_sensor_data, window_size=5):
    """
    กรองสัญญาณดิบ 6 แกน (Ax, Ay, Az, Wx, Wy, Wz) ด้วย Moving Average Filter 5 จุด
    :param raw_sensor_data: numpy array มิติ (N, 6)
    :param window_size: ขนาดหน้าต่างการกรอง (M = 5)
    :return: filtered_data: numpy array มิติ (N, 6) ที่ลดสัญญาณรบกวนแล้ว
    """
    filtered_data = np.zeros_like(raw_sensor_data, dtype=np.float32)
    pad_width = window_size // 2  # สำหรับ 5 จุด pad_width = 2
    kernel = np.ones(window_size) / window_size  # ฟังก์ชันถ่วงน้ำหนักสม่ำเสมอ [0.2, 0.2, 0.2, 0.2, 0.2]

    # ทำการกรองทีละแกน (3 แกนความเร่ง + 3 แกนไจโรสโคป)
    for axis in range(6):
        signal = raw_sensor_data[:, axis]
        # ทำ Edge Padding ที่จุดหัวและท้าย เพื่อไม่ให้ข้อมูลหดสั้นลงและลดผลกระทบ Transient
        padded = np.pad(signal, pad_width, mode='edge')
        # คอนโวลูชันแบบ Discrete Linear Convolution
        smoothed = np.convolve(padded, kernel, mode='valid')
        filtered_data[:, axis] = smoothed[:len(signal)]

    return filtered_data

# ==========================================================
# ตัวอย่างการทดสอบและการแสดงผลเปรียบเทียบ
# ==========================================================
if __name__ == "__main__":
    # สมมุติตัวอย่างสัญญาณความเร่งแกน X ที่มีสัญญาณรบกวนไฟฟ้าความถี่สูง 100 จุดตัวอย่าง (2 วินาที ที่ 50 Hz)
    np.random.seed(42)
    time_ms = np.linspace(0, 2000, 100)
    true_impact = 1.0 + 3.5 * np.exp(-((time_ms - 1000) ** 2) / 10000) # ยอดกระแทก 4.5 G
    noise = np.random.normal(0, 0.25, 100)                             # สัญญาณรบกวนเกาส์เซียน
    raw_signal_6axis = np.zeros((100, 6))
    raw_signal_6axis[:, 0] = (true_impact + noise) * 9.80665           # แกน Ax (m/s^2)

    # นำไปผ่านตัวกรอง Moving Average 5 จุด
    filtered_signal_6axis = apply_moving_average_6axi[illegible]w_signal_6axis, window_size=5)
```

รูปที่ 3.5.1 (ก) ตัวอย่างโค้ต Stratified Train-Test Split

```python
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# X คือ Matrix ฟีเจอร์ 18 มิติ (222, 18)
# y คือ Label 0 (ADL) หรือ 1 (Fall) (222,)

# 1. แบ่งข้อมูลแบบ Stratified 80:20
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,      # ชุด Test 20% (ประมาณ 44-45 ตัวอย่าง)
    random_state=42,     # ล็อก Seed เพื่อให้ผลการทดลองคงเดิม ซ้ำได้ (Reproducible)
    stratify=y           # กระจายสัดส่วน Fall และ ADL ให้เท่ากัน
)

# 2. ปรับสเกลข้อมูล (Standardization)
# สำคัญ: fit เฉพาะ Train แล้วนำไป transform ทั้ง Train และ Test เพื่อป้องกัน Data Leakage
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"ขนาดชุดฝึกสอน (Train): {X_train.shape[0]} ตัวอย่าง (Fall={sum(y_train)}, ADL={len(y_train)-
sum(y_train)})")
print(f"ขนาดชุดทดสอบ (Test):  {X_test.shape[0]} ตัวอย่าง (Fall={sum(y_test)}, ADL={len(y_test)-sum(y_test)})")
```

รูปที่ 3.5.1 (ข) ตัวอย่างโค้ต Stratified Train-Test Split (ต่อ)

ตารางที่ 3.5.1 การแบ่งข้อมูลเพื่อ Stratified Sampling

ชุดข้อมูล	จำนวนตัวอย่างทั้งหมด	ท่าล้ม (Fall)	กิจกรรมปกติ (ADL)	วัตถุประสงค์
Train Set (80%)	178	80	98	ใช้สอนโมเดล MLP ให้เรียนรู้ลักษณะคลื่นสัญญาณ
Test Set (20%)	45	20	25	จำลองเป็นข้อมูลที่ไม่เคยเห็น เพื่อวัด Accuracy, Sensitivity, Specificity
รวมทั้งหมด	223	100	123	ครบถ้วนทุกตัวอย่าง

3.5.2 ดาวน์โหลดข้อมูลดิบจาก CollectData_Web



รูปที่ 3.5.2 รวบรวมไฟล์ข้อมูลดิบจาก CollectData_Web

3.5.2.1 ใช้สคริปต์รัน ในกระบวนการสร้างโมเดล AI

```
def extract_18_features(window_data_6axis):
    """
    สกัดคุณลักษณะ 18 มิติ (18-D Feature Vector) จากหน้าต่างขนาด 50 จุดตัวอย่าง
    :param window_data_6axis: numpy array ขนาด (50, 6) -> Ax, Ay, Az (m/s^2), Gx, Gy, Gz (rad/s)
    :return: feature_vector: list ของทศนิยม 18 ค่า [F0 ถึง F17]
    """
    ax = window_data_6axis[:, 0]
    ay = window_data_6axis[:, 1]
    az = window_data_6axis[:, 2]
    gx = window_data_6axis[:, 3]
    gy = window_data_6axis[:, 4]
    gz = window_data_6axis[:, 5]

    # 1. คำนวณขนาดเวกเตอร์สัพธ์ (Resultant Magnitudes)
    accel_mag = np.sqrt(ax**2 + ay**2 + az**2) / 9.80665  # แปลงเป็นหน่วยแรงโน้มถ่วง (G)
    gyro_mag = np.sqrt(gx**2 + gy**2 + gz**2)             # ขนาดเชิงมุมสัพธ์ (rad/s)
    horiz_mag = np.sqrt(ax**2 + az**2) / 9.80665          # ความเร่งในแนวราบ (G)
    jerk = np.diff(accel_mag)                             # อัตราการเปลี่ยนความเร่ง (G/sample)

    # 2. คำนวณฟีเจอร์ F0 - F17
    f0  = float(np.mean(accel_mag))                                    # F0: ค่าเฉลี่ยความเร่งสัพธ์ (G)
    f1  = float(np.max(accel_mag))                                     # F1: ความเร่งกระแทกสูงสุด A_max
(G)
    f2  = float(np.min(accel_mag))                                     # F2: ความเร่งต่ำสุดช่วงตกอิสระ A_m
(G)
    f3  = float(np.std(accel_mag))                                     # F3: ส่วนเบี่ยงเบนมาตรฐานความเร่ง
    f4  = float(f1 - f2)                                               # F4: พิสัยพลวัต (Dynamic Range
Max - Min)
    f5  = float(np.sum(accel_mag < 0.5) / len(accel_mag))              # F5: สัดส่วนภาวะตกอิสระ (Freefa
Ratio)
    f6  = float(np.sum(accel_mag > 2.5) / len(accel_mag))              # F6: สัดส่วนแรงกระแทกสูง (Impac
Ratio)
    f7  = float(np.mean(gyro_mag))                                     # F7: ค่าเฉลี่ยความเร็วเชิงมุม (rad/
    f8  = float(np.max(gyro_mag))                                      # F8: ความเร็วเชิงมุมสูงสุด w_max
(rad/s)
    f9  = float(np.std(gyro_mag))                                      # F9: ส่วนเบี่ยงเบนความเร็วเชิงมุม
    f10 = float(np.mean(np.abs(ax) + np.abs(ay) + np.abs(az)) / 9.80665)      # F10: พื้นที่ขนาดสัญญาณรวม (SMA
```

รูปที่ 3.5.2.1 (ก) ตัวอย่างฟังก์ชั่นการสกัดคุณลักษณะ 18 มิติ

```
    # 1. คำนวณขนาดเวกเตอร์สัพธ์ (Resultant Magnitudes)
    accel_mag = np.sqrt(ax**2 + ay**2 + az**2) / 9.80665  # แปลงเป็นหน่วยแรงโน้มถ่วง (G)
    gyro_mag = np.sqrt(gx**2 + gy**2 + gz**2)             # ขนาดเชิงมุมสัพธ์ (rad/s)
    horiz_mag = np.sqrt(ax**2 + az**2) / 9.80665          # ความเร่งในแนวราบ (G)
    jerk = np.diff(accel_mag)                             # อัตราการเปลี่ยนความเร่ง (G/sample)

    # 2. คำนวณฟีเจอร์ F0 - F17
    f0  = float(np.mean(accel_mag))                                    # F0: ค่าเฉลี่ยความเร่งสัพธ์ (G)
    f1  = float(np.max(accel_mag))                                     # F1: ความเร่งกระแทกสูงสุด A_max
(G)
    f2  = float(np.min(accel_mag))                                     # F2: ความเร่งต่ำสุดช่วงตกอิสระ A_min
(G)
    f3  = float(np.std(accel_mag))                                     # F3: ส่วนเบี่ยงเบนมาตรฐานความเร่ง
    f4  = float(f1 - f2)                                               # F4: พิสัยพลวัด (Dynamic Range =
Max - Min)
    f5  = float(np.sum(accel_mag < 0.5) / len(accel_mag))              # F5: สัดส่วนภาวะตกอิสระ (Freefall
Ratio)
    f6  = float(np.sum(accel_mag > 2.5) / len(accel_mag))              # F6: สัดส่วนแรงกระแทกสูง (Impact
Ratio)
    f7  = float(np.mean(gyro_mag))                                     # F7: ค่าเฉลี่ยความเร็วเชิงมุม (rad/s)
    f8  = float(np.max(gyro_mag))                                      # F8: ความเร็วเชิงมุมสูงสุด w_max
(rad/s)
    f9  = float(np.std(gyro_mag))                                      # F9: ส่วนเบี่ยงเบนความเร็วเชิงมุม
    f10 = float(np.mean(np.abs(ax) + np.abs(ay) + np.abs(az)) / 9.80665)  # F10: พื้นที่ขนาดสัญญาณรวม (SMA,
G)
    f11 = float(np.mean(horiz_mag))                                    # F11: ค่าเฉลี่ยความเร่งแนวราบ (G)
    f12 = float(np.max(horiz_mag))                                     # F12: ความเร่งแนวราบสูงสุด (G)
    f13 = float(np.mean(np.abs(jerk)) if len(jerk) > 0 else 0.0)       # F13: ค่าเฉลี่ย Jerk
    f14 = float(np.max(np.abs(jerk)) if len(jerk) > 0 else 0.0)        # F14: Jerk สูงสุด (Peak Jerk)
    f15 = float(np.sum(np.diff(np.sign(accel_mag - 1.0)) != 0))        # F15: อัตราการตัดผ่าน 1.0 G (ZCR)

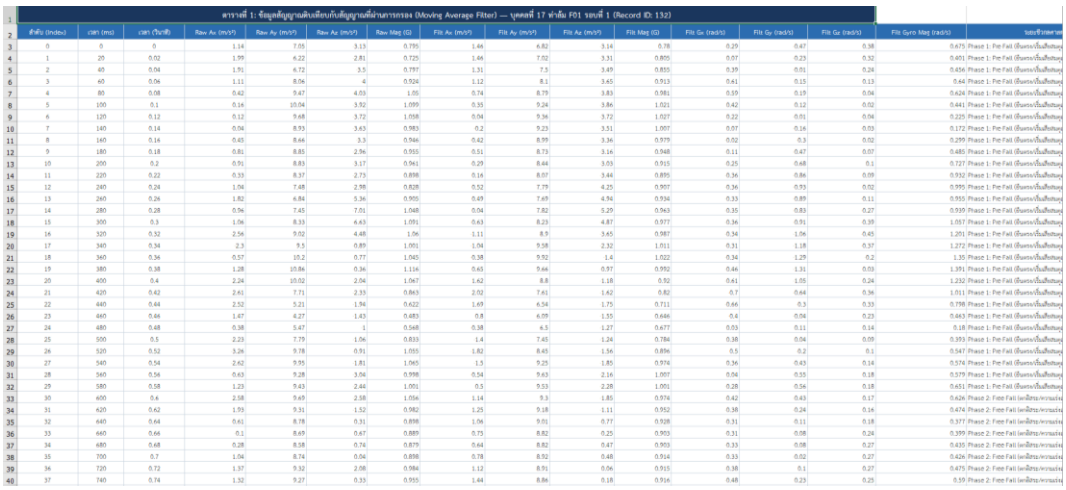
    # สหสัมพันธ์แบบเพียร์สันระหว่างแกน (Cross-Axis Correlation)
    std_x, std_y, std_z = np.std(ax), np.std(ay), np.std(az)
    f16 = float(np.corrcoef(ax, ay)[0, 1] if std_x > 1e-4 and std_y > 1e-4 else 0.0)  # F16: Corr(X, Y)
    f17 = float(np.corrcoef(ay, az)[0, 1] if std_y > 1e-4 and std_z > 1e-4 else 0.0)  # F17: Corr(Y, Z)

    # จัดการกรณีค่าว่างทางคณิตศาสตร์ (NaN Handing)
    if np.isnan(f16): f16 = 0.0
    if np.isnan(f17): f17 = 0.0
```

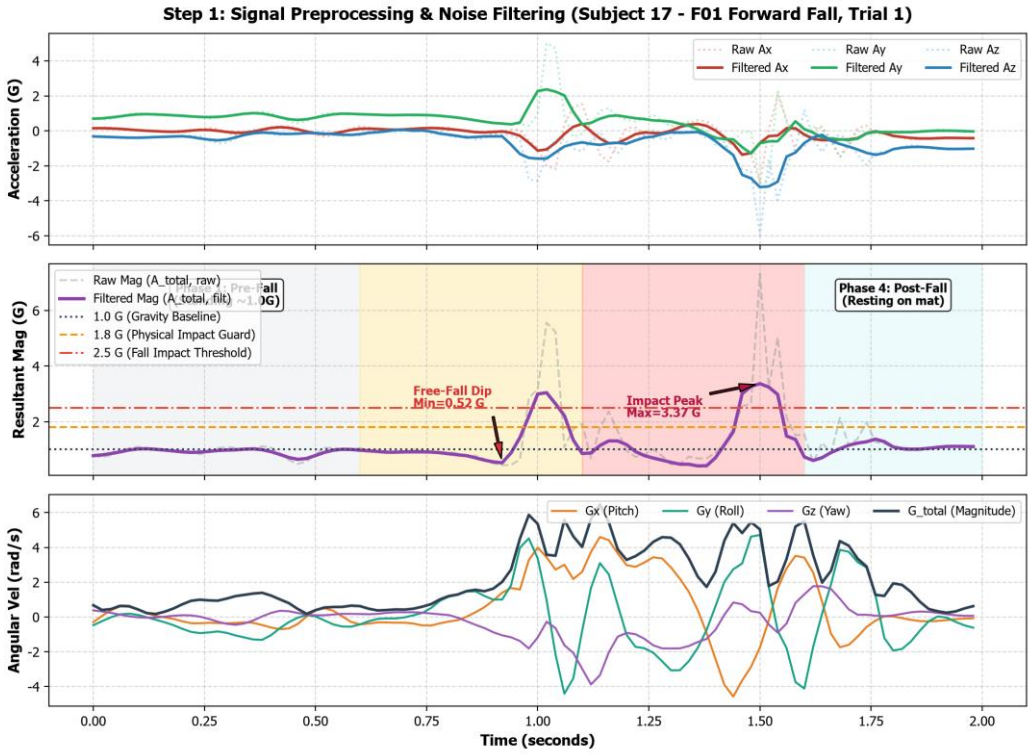
รูปที่ 3.5.2.1 (ข) ตัวอย่างฟังก์ชั่นการสกัดคุณลักษณะ 18 มิติ (ต่อ)

3.5.2.2 การรองสัญญารดิบ

สัญญาณดิบ 6 แกน ($A_x, A_y, A_z, \omega_x, \omega_y, \omega_z$) ถูกนำมาผ่าน Moving Average Filter ขนาด 5 จุด เพื่อกำจัดสัญญาณรบกวนความถี่สูงทางกลและคลื่นรบกวนทางไฟฟ้า โดยยังคงแอมพลิจูดความชันของแรงกระแทกไว้อย่างสมบูรณ์



รูปที่ 3.5.2.2 (ก) ตัวอย่างข้อมูลสัญญาณดิบเทียบกับสัญญาณที่ผ่านการกรอง (Moving Average Filter) บุคคลที่ 17 ท่าล้ม F01 รอบที่ 1



รูปที่ 3.5.2.2 (ข) การกรองสัญญาณและการระบุระยะชีวกลศาสตร์การล้มไปข้างหน้าจากอาสาสมัครที่ 17 รอบที่ 1

ตารางที่ 3.5.2.2 ตัวอย่างข้อมูลสัญญาณดิบเทียบกับสัญญาณที่ผ่านการกรองใน 4 ระยะชีวกลศาสตร์ อาสาสมัครที่ 17 ท่า F01 รอบที่ 1

เวลา (ms)	Raw A_{total}(G)	Filt A_{total}(G)	Filt G_{total}(rad/s)	สภาพการเคลื่อนไหว	ระยะชีวกลศาสตร์ (Biomechanical Phase)
0	0.80 G	0.78 G	0.68 rad/s	ยืนตรงนิ่ง	**Phase 1: Pre-Fall** (ยืนตรง/เริ่มเสียสมดุล)
300	1.09 G	0.98 G	1.06 rad/s	ลำตัวเริ่มเอียงไปข้างหน้า	**Phase 1: Pre-Fall** (แรงโน้มถ่วงปกติ ≈ 1 G)
600	1.06 G	0.97 G	0.63 rad/s	เสียการทรงตัวสมบูรณ์	**Phase 2: Free-Fall** (เริ่มทิ้งตัวลง)
900	0.50 G	0.55 G	1.63 rad/s	สภาพไร้น้ำหนักชั่วขณะ	**Phase 2: Free-Fall Dip** (จุดต่ำสุด A_{min} = 0.41 G)
1,100	1.93 G	0.86 G	**4.03 rad/s**	ลำตัวหมุนเหวี่ยงลงสู่พื้น	**Phase 3: Impact Transition** (ความเร็วเชิงมุมพุ่งสูง)
1,500	7.33 G	3.37 G	5.04 rad/s	กระทบเบาะนิรภัย	**Phase 3: Impact Peak** (ยอดแรงกระแทก $A_{max} =$ 3.37 G)
1,980	1.09 G	1.10 G	0.62 rad/s	ร่างกายนอนราบหยุดนิ่ง	**Phase 4: Post-Fall Rest** (แกน Z รับแรงโน้มถ่วง 1 G)

3.5.2.3 การสกัดคุณลักษณะ 18 มิติ

ตาราง 3.5.2.3 รายละเอียดฟังก์ชั่นการสกัดคุณลักษณะ 18 มิติ

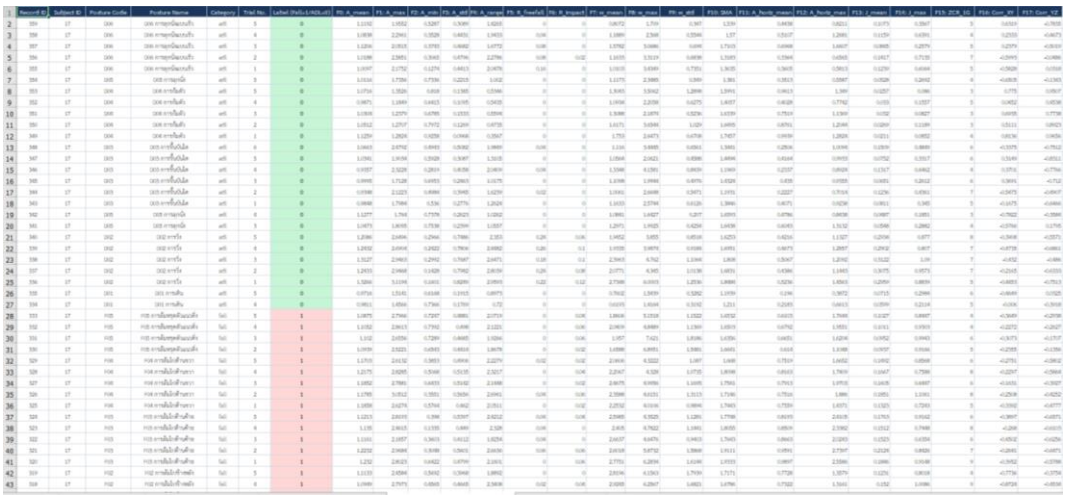
มิติ	สัญลักษณ์	ชื่อคุณลักษณะ (Feature Name)	สูตรการคำนวณ	หน่วย	ความหมายต่อการจำแนก
0	A_{mean}	ค่าเฉลี่ยขนาดความเร่งลัพธ์	$\frac{1}{N}\sum A_k$	G	ระดับความเร่งเฉลี่ยในหน้าต่าง

ตาราง รายละเอียดฟังก์ชั่นการสกัดคุณลักษณะ 18 มิติ (ต่อ)

มิติ	สัญลักษณ์	ชื่อคุณลักษณะ (Feature Name)	สูตรการคำนวณ	หน่วย	ความหมายต่อการจำแนก
1	$A_{\max}$	ค่าสูงสุดขนาดความเร่ง (แรงกระแทก)	$\max(A_k)$	G	จุดกระแทกพื้นสูงสุด เกินเกณฑ์ 2.5 G
2	$A_{\min}$	**ค่าต่ำสุดขนาดความเร่ง (ตกอิสระ)**	$\min(A_k)$	G	จุดตกอิสระ ต่ำกว่าระดับปกติ (<0.5G)
3	A_{std}	ส่วนเบี่ยงเบนมาตรฐานความเร่ง	$\sigma(A_k)$	G	ความแปรปรวนสูงจากแรงกระแทก
4	A_{range}	พิสัยขนาดความเร่ง	$A_{\max} - A_{\min}$	G	ความกว้างของคลื่นแรงกระแทก
5	R_{freefall}	**อัตราส่วนการตกอิสระ**	$\frac{1}{N}\sum[A_k < 0.5]$	ratio	มีช่วงเวลาตกอิสระคิดเป็น 8% ของหน้าต่าง
6	R_{impact}	**อัตราส่วนแรงกระแทก**	$\frac{1}{N}\sum[A_k > 2.5]$	ratio	มีช่วงแรงกระแทกสูงคิดเป็น 16% ของหน้าต่าง
7	ω_{mean}	ค่าเฉลี่ยความเร็วเชิงมุมไจโร	$\frac{1}{N}\sum G_k$	rad/s	ลำตัวมีการหมุนเฉลี่ยสูง
8	$\omega_{\max}$	**ค่าสูงสุดความเร็วเชิงมุมไจโร**	$\max(G_k)$	rad/s	ลำตัวเหวี่ยงไปข้างหน้าอย่างรวดเร็ว
9	ω_{std}	ส่วนเบี่ยงเบนมาตรฐานไจโร	$\sigma(G_k)$	rad/s	ความแปรปรวนของการหมุนลำตัว

ตาราง รายละเอียดฟังก์ชั่นการสกัดคุณลักษณะ 18 มิติ (ต่อ)

มิติ	สัญลักษณ์	ชื่อคุณลักษณะ (Feature Name)	สูตรการคำนวณ	หน่วย	ความหมายต่อการจำแนก
10	SMA	พื้นที่ขนาดสัญญาณรวม	$\frac{1}{N}\sum(\mid A_x \mid + \mid A_y \mid + \mid A_z \mid)$	G	พลังงานจลน์รวมของการเคลื่อนไหว
11	$A_{\text{horiz, mean}}$	ค่าเฉลี่ยความเร่งแนวราบ	$\frac{1}{N}\sum\sqrt{A_x^2 + A_z^2}$	G	การเคลื่อนที่ในแนวระนาบเฉลี่ย
12	$A_{\text{horiz, max}}$	ค่าสูงสุดความเร่งแนวราบ	$\max\left(\sqrt{A_x^2 + A_z^2}\right)$	G	แรงกระแทกที่แผ่ไปในระนาบพื้น
13	J_{mean}	ค่าเฉลี่ยอัตราเปลี่ยนความเร่ง (Jerk)	$\frac{1}{N-1}\sum \mid \Delta A_k \mid$	G/sample	การกระชากตัวเฉลี่ย
14	J_{max}	**ค่าสูงสุดการกระชากตัว (Peak Jerk)**	$\max(\mid \Delta A_k \mid)$	G/sample	แรงกระชากขณะกระทบเบาะ
15	ZCR_{1G}	อัตราการตัดผ่านระดับ 1.0 G	นับครั้งที่ตัดผ่าน 1.0	ครั้ง	คลื่นแกว่งข้ามแรงโน้มถ่วง 6 ครั้ง
16	r_{xy}	สหสัมพันธ์แกน X และ Y	$\text{Pearson}(A_x, A_y)$	coeff	ความสัมพันธ์แนวข้างและแนวดิ่ง
17	r_{yz}	สหสัมพันธ์แกน Y และ Z	$\text{Pearson}(A_y, A_z)$	coeff	การถ่ายเทแรงจากแนวดิ่งสู่แนวหน้า-หลัง



รูปที่ 3.5.2.3 (ก) Features Dataset

ตารางที่ 2: ค่าคุณลักษณะ 18 มิติที่สกัดได้จากหน้าต่างสัญญาณ 1.0 วินาที — บุคคลที่ 17 ท่าล้ม F01 รอบที่ 1

มิติที่ (Dim)	กลุ่มคุณลักษณะ (Feature Group)	ชื่อคุณลักษณะ (Feature Name)	สัญลักษณ์ (Symbol)	สูตรการคำนวณ (Formula)	หน่วย (Unit)	ค่าที่คำนวณได้ (Extracted Value)
คุณลักษณะที่ 0	กลุ่มขนาดความแรงสัดส่วน	ค่าเฉลี่ยขนาดความแรงสัดส่วน (Mean Acceleration)	A_mean	mean(A_total)	G	1.3486
คุณลักษณะที่ 1	กลุ่มขนาดความแรงสัดส่วน	ค่าสูงสุดขนาดความแรง (แรงกระแทก) (Peak Impact Acceleration)	A_max	max(A_total)	G	3.3683
คุณลักษณะที่ 2	กลุ่มขนาดความแรงสัดส่วน	ค่าต่ำสุดขนาดความแรง (ระยะลอยอิสระ) (Lowest Freefall Dip)	A_min	min(A_total)	G	0.4093
คุณลักษณะที่ 3	กลุ่มขนาดความแรงสัดส่วน	ส่วนเบี่ยงเบนมาตรฐานขนาดความแรง (Std Dev Acceleration)	A_std	std(A_total)	G	0.8225
คุณลักษณะที่ 4	กลุ่มขนาดความแรงสัดส่วน	พิสัยขนาดความแรง (Max - Min) (Dynamic Range)	A_range	max(A) - min(A)	G	2.959
คุณลักษณะที่ 5	กลุ่มพิวเกตศาสตร์การล้ม	อัตราส่วนการลอยอิสระ (A < 0.5 G) (Freefall Ratio)	R_freefall	count(A < 0.5G) / N	ratio	0.08
คุณลักษณะที่ 6	กลุ่มพิวเกตศาสตร์การล้ม	อัตราส่วนแรงกระแทก (A > 2.5 G) (Impact Ratio)	R_impact	count(A > 2.5G) / N	ratio	0.16
คุณลักษณะที่ 7	กลุ่มความเร็วเชิงมุมไจโรสโคป	ค่าเฉลี่ยขนาดความเร็วเชิงมุม (Mean Angular Velocity)	w_mean	mean(G_total)	rad/s	3.2676
คุณลักษณะที่ 8	กลุ่มความเร็วเชิงมุมไจโรสโคป	ค่าสูงสุดขนาดความเร็วเชิงมุม (Peak Angular Velocity)	w_max	max(G_total)	rad/s	6.4806
คุณลักษณะที่ 9	กลุ่มความเร็วเชิงมุมไจโรสโคป	ส่วนเบี่ยงเบนมาตรฐานความเร็วเชิงมุม (Std Dev Angular Velocity)	w_std	std(G_total)	rad/s	1.7075
คุณลักษณะที่ 10	กลุ่มพลังงานและการเคลื่อนที่ราบ	พื้นที่ขนาดสัญญาณรวม (SMA) (Signal Magnitude Area)	SMA	mean(|Ax| + |Ay| + |Az|)	G	1.9781
คุณลักษณะที่ 11	กลุ่มพลังงานและการเคลื่อนที่ราบ	ค่าเฉลี่ยขนาดความแรงในแนวราบ (Mean Horizontal Accel)	A_horiz_mean	mean(sqrt(Ax² + Az²))	G	1.136
คุณลักษณะที่ 12	กลุ่มพลังงานและการเคลื่อนที่ราบ	ค่าสูงสุดขนาดความแรงในแนวราบ (Peak Horizontal Accel)	A_horiz_max	max(sqrt(Ax² + Az²))	G	3.2917
คุณลักษณะที่ 13	กลุ่มอัตราการเปลี่ยนความแรง (Jerk)	ค่าเฉลี่ยอัตราการเปลี่ยนความแรง (Jerk) (Mean Jerk)	J_mean	mean(|diff(A)|)	G/sample	0.2154
คุณลักษณะที่ 14	กลุ่มอัตราการเปลี่ยนความแรง (Jerk)	ค่าสูงสุดอัตราการเปลี่ยนความแรง (Peak Jerk) (Peak Jerk)	J_max	max(|diff(A)|)	G/sample	1.4983
คุณลักษณะที่ 15	กลุ่มความถี่และรอบสมดุล	อัตราการตัดผ่านระดับเฉลี่ยในช่วง 1.0 G (Zero-Crossing Rate (1.0G))	ZCR_1G	count(sign_change(A - 1.0G))	count	6
คุณลักษณะที่ 16	กลุ่มสหสัมพันธ์ข้ามแกน	สหสัมพันธ์ระหว่างแกน X และ Y (Pearson Corr (X-Y))	Cor_XY	Pearson_Cor(Ax, Ay)	coeff	0.0122
คุณลักษณะที่ 17	กลุ่มสหสัมพันธ์ข้ามแกน	สหสัมพันธ์ระหว่างแกน Y และ Z (Pearson Cor (Y-Z))	Cor_YZ	Pearson_Cor(Ay, Az)	coeff	0.2841

รูปที่ 3.5.2.3 (ข) ตัวอย่าง Features จากท่าล้มไปข้างหน้า 1 ครั้ง ของอาสาสมัครที่ 17

การรวบรวมข้อมูลและการกระจายตัวเปรียบเทียบ Fall vs. ADL

เมื่อประมวลผลคุณลักษณะ 18 มิติครบทั้ง 223 รอบการทดลอง (Fall 100 รอบ + ADL 123 รอบ จากอาสาสมัคร) จะเห็นการแยกกลุ่มของข้อมูล (Feature Space Clustering) อย่างชัดเจน

```python
import pandas as pd
import matplotlib.pyplot as plt

def plot_fall_vs_adl_distribution(df_dataset):
    """
    พล็อตกราฟ Scatter Matrix 4 มิติเพื่อแสดงการแยกตัวของคลาส Fall vs ADL
    :param df_dataset: DataFrame ที่มีคอลัมน์ f1_A_max, f2_A_min, f5_R_freefall,
                       f6_R_impact, f8_w_max, f10_SMA, f14_J_max, label (1=Fall, 0=ADL)
    """
    falls_df = df_dataset[df_dataset['label'] == 1]
    adls_df  = df_dataset[df_dataset['label'] == 0]

    fig, axs = plt.subplots(2, 2, figsize=(12, 10))
    plt.rcParams['font.sans-serif'] = ['Tahoma', 'DejaVu Sans']

    # (A) แรงกระแทกสูงสุด vs สัดส่วนช่วงตกอิสระ (A_max vs Freefall Ratio)
    axs[0, 0].scatter(adls_df['f1_A_max'], adls_df['f5_R_freefall'], color='#3498db', alpha=0.7, label='ADL
(กิจวัตรปกติ)')
    axs[0, 0].scatter(falls_df['f1_A_max'], falls_df['f5_R_freefall'], color='#e74c3c', marker='^',
alpha=0.8, edgecolors='black', s=55, label='Fall (การหกล้ม)')
    axs[0, 0].set_xlabel('Peak Impact Acceleration A_max (G)', fontweight='bold')
    axs[0, 0].set_ylabel('Freefall Ratio R_freefall', fontweight='bold')
    axs[0, 0].set_title('(A) จุดกระแทกสูงสุดเทียบกับช่วงตกอิสระ', fontweight='bold')
    axs[0, 0].grid(True, linestyle='--', alpha=0.5)
    axs[0, 0].legend()
```

รูปที่ 3.5.2.3 (ค) ตัวอย่างโค้ดส่วนการรวบรวมข้อมูลและการกระจายตัวเปรียบเทียบ train_model_collectdata.py

```
    axs[0, 0].legend()

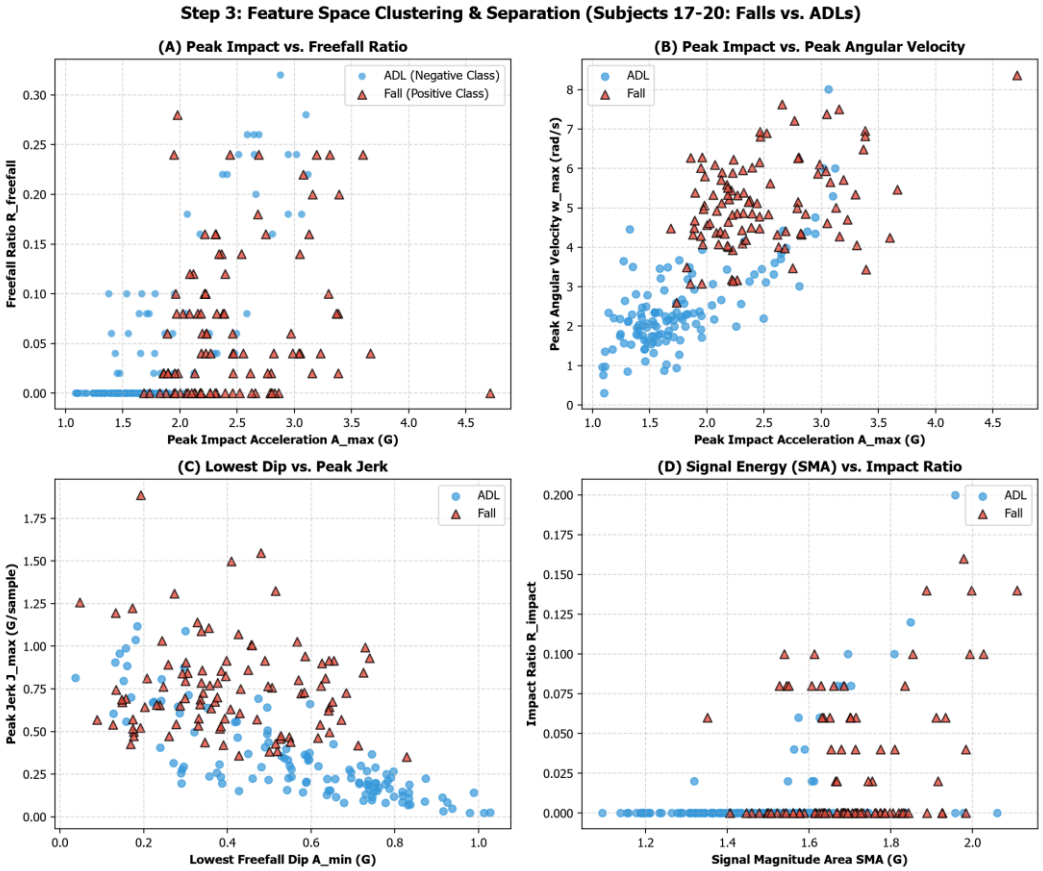
    # (B) แรงกระแทกสูงสุด vs ความเร็วเชิงมุมสูงสุด (A_max vs w_max)
    axs[0, 1].scatter(adls_df['f1_A_max'], adls_df['f8_w_max'], color='#3498db', alpha=0.7, label='ADL')
    axs[0, 1].scatter(falls_df['f1_A_max'], falls_df['f8_w_max'], color='#e74c3c', marker='^', alpha=0.8,
edgecolors='black', s=55, label='Fall')
    axs[0, 1].set_xlabel('Peak Impact Acceleration A_max (G)', fontweight='bold')
    axs[0, 1].set_ylabel('Peak Angular Velocity w_max (rad/s)', fontweight='bold')
    axs[0, 1].set_title('(B) จุดกระแทกสูงสุดเทียบกับความเร็วเชิงมุม', fontweight='bold')
    axs[0, 1].grid(True, linestyle='--', alpha=0.5)
    axs[0, 1].legend()

    # (C) ค่าต่ำสุดช่วงตกอิสระ vs Jerk สูงสุด (A_min vs Peak Jerk)
    axs[1, 0].scatter(adls_df['f2_A_min'], adls_df['f14_J_max'], color='#3498db', alpha=0.7, label='ADL')
    axs[1, 0].scatter(falls_df['f2_A_min'], falls_df['f14_J_max'], color='#e74c3c', marker='^', alpha=0.8,
edgecolors='black', s=55, label='Fall')
    axs[1, 0].set_xlabel('Lowest Freefall Dip A_min (G)', fontweight='bold')
    axs[1, 0].set_ylabel('Peak Jerk J_max (G/sample)', fontweight='bold')
    axs[1, 0].set_title('(C) ช่วงตกอิสระต่ำสุดเทียบกับ Jerk สูงสุด', fontweight='bold')
    axs[1, 0].grid(True, linestyle='--', alpha=0.5)
    axs[1, 0].legend()

    # (D) พื้นที่ขนาดสัญญาณรวม vs สัดส่วนแรงกระแทก (SMA vs Impact Ratio)
    axs[1, 1].scatter(adls_df['f10_SMA'], adls_df['f6_R_impact'], color='#3498db', alpha=0.7, label='ADL')
    axs[1, 1].scatter(falls_df['f10_SMA'], falls_df['f6_R_impact'], color='#e74c3c', marker='^', alpha=0.8,
edgecolors='black', s=55, label='Fall')
    axs[1, 1].set_xlabel('Signal Magnitude Area SMA (G)', fontweight='bold')
    axs[1, 1].set_ylabel('Impact Ratio R_impact', fontweight='bold')
    axs[1, 1].set_title('(D) พลังงานคลื่นสัญญาณ (SMA) เทียบกับสัดส่วนแรงกระแทก', fontweight='bold')
    axs[1, 1].grid(True, linestyle='--', alpha=0.5)
    axs[1, 1].legend()

    plt.suptitle('การกระจายตัวของฟีเจอร์ในปริภูมิคุณลักษณะ (Feature Space: Falls vs. ADLs)', fontsize=14,
fontweight='bold')
    plt.tight_layout()
    plt.show()
```

รูปที่ 3.5.2.3 (ง) ตัวอย่างโค้ตส่วนการรวบรวมข้อมูลและการกระจายตัวเปรียบเทียบ train_model_collectdata.py



รูปที่ 3.5.2.3 (จ) การกระจายตัวของคุณลักษณะเปรียบเทียบระหว่าง Fall และ ADL

3.5.2.4 การเทรนโมเดล TinyML และการทดสอบผล

หลังจากเสร็จสิ้นขั้นตอนการสกัดคุณลักษณะเชิงลึก 18 มิติ จากแฟ้มข้อมูลสัญญาณเซ็นเซอร์ MPU-6050 ทั้งสิ้น 223 แฟ้มบันทึก (ประกอบด้วยเหตุการณ์การหกล้มจริง 100 ตัวอย่าง และกิจกรรมประจำวันปกติ 123 ตัวอย่าง จากกลุ่มอาสาสมัคร) ข้อมูลคุณลักษณะดังกล่าวจะถูกนำเข้าสู่กระบวนการฝึกสอนแบบจำลองปัญญาประดิษฐ์ฝังตัว (Tiny Machine Learning: TinyML) เพื่อพัฒนาโครงข่ายประสาทเทียมที่มีความแม่นยำสูง มีขนาดกะทัดรัด และสามารถประมวลผลบนไมโครคอนโทรลเลอร์ ESP32 ได้อย่างมีประสิทธิภาพ โดยมีขั้นตอนการดำเนินงานดังนี้

```python
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score, recall_score

def train_and_evaluate_mlp(X, y):
    """
    เทรนโครงข่ายประสาทเทียม Multi-Layer Perceptron (18 -> 32 -> 16 -> 1)
    :param X: แมทริกซ์ฟีเจอร์ 18 มิติ ขนาด (222, 18)
    :param y: เวกเตอร์เป้าหมาย ขนาด (222,) โดย 1=Fall, 0=ADL
    """
    # 1. แบ่งชุดข้อมูลแบบ Stratified Sampling 80:20 (Train 178, Test 44 ตัวอย่าง)
    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=0.20,
        random_state=42,
        stratify=y
    )

    # 2. ปรับสเกลข้อมูลด้วย Z-Score Standardization (Fit เฉพาะ Train เพื่อป้องกัน Data Leakage)
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled  = scaler.transform(X_test)

    # 3. กำหนดสถาปัตยกรรม MLP: 18 -> 32 -> 16 -> 1
    # Hidden Layer 1 = 32 โหนด (ReLU), Hidden Layer 2 = 16 โหนด (ReLU), Output = 1 โหนด (Sigmoid/Log-loss)
    mlp = MLPClassifier(
        hidden_layer_sizes=(32, 16),
        activation='relu',
        solver='adam',
        alpha=0.001,              # L2 Regularization เพื่อป้องกัน Overfitting บนข้อมูลขนาด 222 ตัวอย่าง
        learning_rate_init=0.002,
        max_iter=800,
        random_state=42,
        early_stopping=False
```

รูปที่ 3.5.2.4 (ก) สคริปต์การเทรนโมเดล TinyML

```
    # 4. ทำการฝึกสอนโมเดล (Training)
    print("กำลังเริ่มฝึกสอนโมเดล MLP...")
    mlp.fit(X_train_scaled, y_train)

    # 5. การทดสอบและการประเมินผลบน Test Set
    y_pred = mlp.predict(X_test_scaled)
    y_prob = mlp.predict_proba(X_test_scaled)[:, 1]

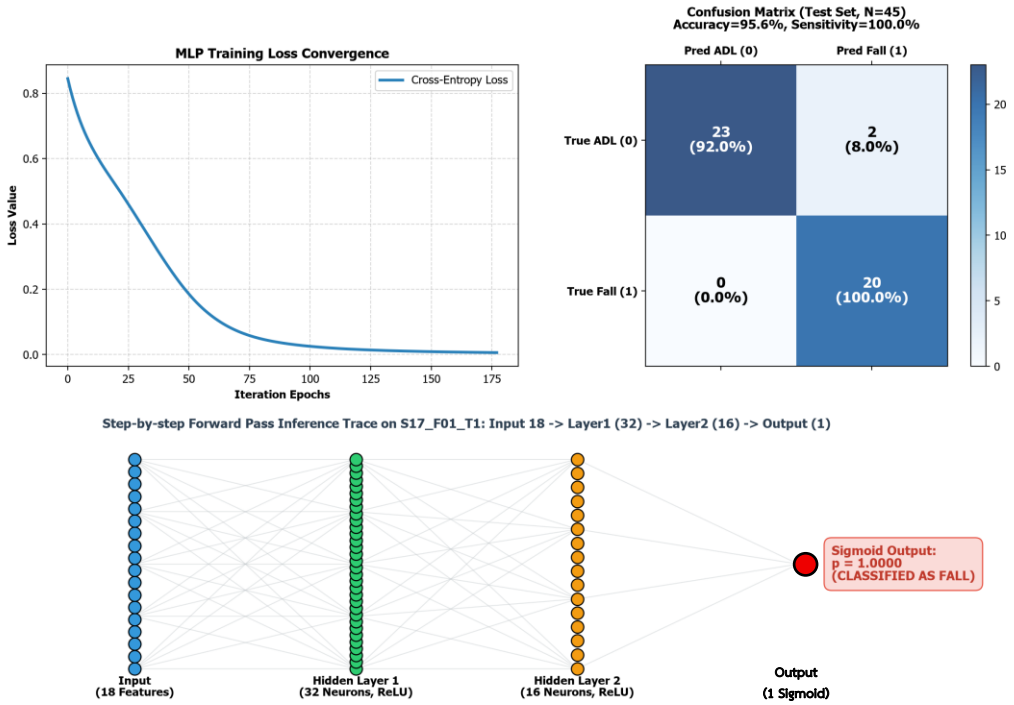
    acc  = accuracy_score(y_test, y_pred)
    sens = recall_score(y_test, y_pred) # ค่าความไวต่อท่าล้ม (Sensitivity / Recall)
    cm   = confusion_matrix(y_test, y_pred)
    tn, fp, fn, tp = cm.ravel()
    spec = tn / (tn + fp)               # ค่าความจำเพาะต่อท่าปกติ (Specificity)

    print("\n" + "="*50)
    print("ผลการประเมินประสิทธิภาพโมเดล MLP (18 -> 32 -> 16 -> 1):")
    print(f"  ความถูกต้องโดยรวม (Accuracy): {acc * 100:.2f}%")
    print(f"  ความไวต่อการล้ม (Sensitivity / Recall): {sens * 100:.2f}% (ตรวจจับล้มได้ {tp}/{tp+fn})")
    print(f"  ความจำเพาะต่อท่าปกติ (Specificity):   {spec * 100:.2f}% (ไม่แจ้งเตือนหลอก {tn}/{tn+fp})")
    print("\nConfusion Matrix:")
    print(f"  [[TN={tn}, FP={fp}],")
    print(f"   [FN={fn}, TP={tp}]]")
    print("="*50)

    # 6. จำลองการคำนวณแบบ Forward Pass ในภาษา C (TinyML Inference บน ESP32)
    sample_input = X_test_scaled[0:1] # ตัวอย่าง 1 แถว
    # ชั้นที่ 1: Dense 18 -> 32 + ReLU
    h1 = np.maximum(0, np.dot(sample_input, mlp.coefs_[0]) + mlp.intercepts_[0])
    # ชั้นที่ 2: Dense 32 -> 16 + ReLU
    h2 = np.maximum(0, np.dot(h1, mlp.coefs_[1]) + mlp.intercepts_[1])
    # ชั้นส่งออก: Dense 16 -> 1 + Sigmoid
    z_out = np.dot(h2, mlp.coefs_[2]) + mlp.intercepts_[2]
    prob_fall = 1.0 / (1.0 + np.exp(-z_out[0, 0]))

    print(f"\nทดสอบจำลอง Forward Pass (TinyML Emulation):")
    print(f"  ความน่าจะเป็นที่จะล้ม: {prob_fall * 100:.2f}%  ผลทำนาย: {'หกล้ม (FALL)' if prob_fall >= 0.5 e
'ปกติ (ADL)'}")
```

รูปที่ 3.5.2.4 (ข) สคริปต์การเทรนโมเดล TinyML (ต่อ)



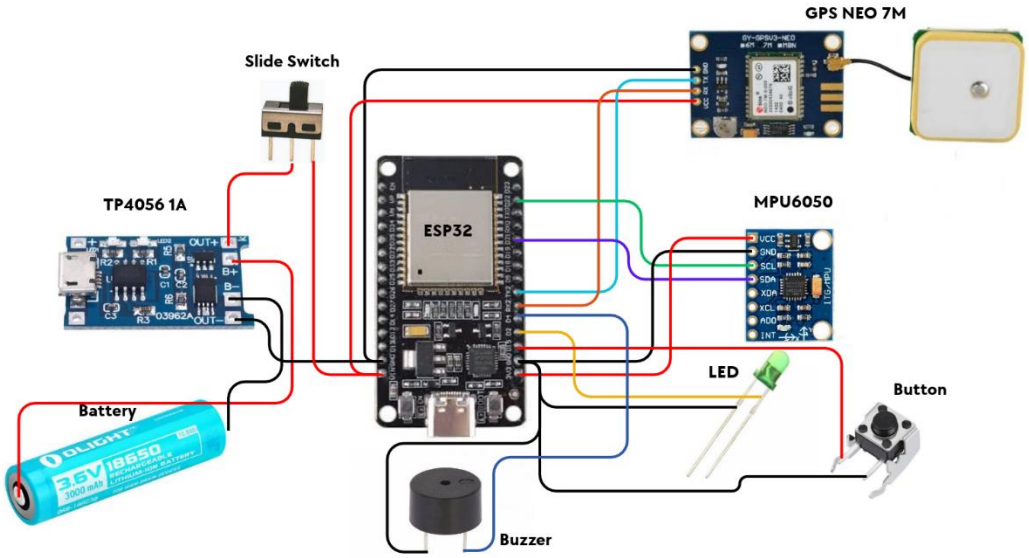
รูปที่ 3.5.2.4 (ค) กราฟการฝึกสอนโมเดล เมทริกซ์ความสับสน และการจำลอง Forward Pass

ตารางที่ 3.5.2.4 เมทริกซ์ความสับสน (Confusion Matrix บนชุดข้อมูลทดสอบ 45 ตัวอย่าง)

เหตุการณ์จริง \ ผลทำนาย	ทำนายเป็น ADL (Pred 0)	ทำนายเป็น Fall (Pred 1)	รวมเหตุการณ์จริง	อัตราความถูกต้องรายคลาส
กิจกรรมประจำวันจริง	**23**	**2**	25	92% (Specificity)
การหกล้มจริง	**0**	**20**	20	100%(Sensitivity)
รวมผลทำนาย	**23**	**22**	**45**	**Accuracy = 95.56%**

3.6 เตรียมอุปกรณ์ฮาร์ดแวร์ สำหรับลงโมเดลAI

โดยอุปกรณ์ฮาร์ดแวร์จะถูกพัฒนาเพิ่มจากเดิมโดยเพิ่ม GPS (NEO 7M) เพื่อเตรียมรองรับการส่งตำแหน่งผู้ส่วมใส่ ทั้งในและนอกอาคาร



รูปที่ 3.6 วงจร Hardware_CollectDataAll เบื้องต้น

ตารางที่ 3.6 รายละเอียดการเชื่อมต่อขาสัญญาณของ Hardware_CollectDataAll

ชิ้นส่วนฮาร์ดแวร์	**ขาบนโมดูล (Pin Name)**	**ขาบน ESP32**	**ชนิดสัญญาณ**	**ฟังก์ชันการทำงาน**
MPU-6050	VCC	3.3V	Power	ไฟเลี้ยงโมดูลความเร่งและไจโรสโคป
	GND	GND	Ground	กราวด์ร่วมของระบบ
	SDA	GPIO 21 (D21)	Digital I/O	สายส่งข้อมูล I2C Data (Wire.begin(21, 22))
	SCL	GPIO 22 (D22)	Digital Output	สายสัญญาณนาฬิกา I2C Clock (ความเร็ว 400 kHz)
	AD0	GND	Logic Low	ล็อก Address I2C ประจำอุปกรณ์เป็น 0x68
NEO-7M	VCC	5V (VIN) / 3.3V	Power (5V DC)	ไฟเลี้ยงโมดูลรับสัญญาณดาวเทียม
	GND	GND	Ground (0V)	กราวด์ร่วมของระบบ
	TXD (GPS Transmit)	GPIO 16 (RX2)	Serial Data	โมดูล GPS ส่งข้อมูลพิกัด NMEA เข้า ESP32
	RXD (GPS Receive)	GPIO 17 (TX2)	Serial Data	ESP32 ส่งคำสั่งควบคุมไปยังโมดูล GPS (9600 bps)
Buzzer	(+) / I/O	GPIO 5 (D5)	Digital Output	ส่งสัญญาณ High ขับเสียงไซเรนเตือนภัย / เสียงบี๊บ

ตารางที่ 3.6 รายละเอียดการเชื่อมต่อขาสัญญาณของ Hardware_CollectDataAll (ต่อ)

ชิ้นส่วนฮาร์ดแวร์	**ขาบนโมดูล (Pin Name)**	**ขาบน ESP32**	**ชนิดสัญญาณ**	**ฟังก์ชันการทำงาน**
	(-)	GND	Ground	กราวด์ร่วมของระบบ
Tactile Push Button	ขา 1 (Signal)	GPIO 18 (D18)	Digital Input	Active LOW: ปกติ HIGH / กดปุ่มดึงลงเป็น LOW
	ขา 2 (Ground)	GND	Ground (0V)	ต่อเข้ากราวด์ร่วม
Status LED	Onboard LED	GPIO 2 (D2)	Digital Output	แสดงสถานะการสุ่มอ่านสัญญาณและ Wi-Fi
Battery & Power	OUT+	VIN (5V)	Power (5.0V)	ไฟเลี้ยงบอร์ดหลักจากวงจร Boost Converter
	OUT-	GND	Ground	กราวด์ร่วมของแบตเตอรี่

3.7 สร้างเว็บแอพสำหรับติดตามผู้สวมใส่ครบวงจร (SoftwarePHP)

เพื่อนำผลลัพธ์จากการประมวลผลโมเดลปัญญาประดิษฐ์บนอุปกรณ์สวมใส่ไปประยุกต์ใช้งานจริงในการดูแลผู้สูงอายุและผู้ป่วยกลุ่มเสี่ยง ผู้จัดทำโครงงานได้ออกแบบและพัฒนาระบบเว็บแอปพลิเคชันส่วนกลางสำหรับติดตามและแจ้งเตือนภัยฉุกเฉินแบบครบวงจร ภายใต้ไดเรกทอรี CareGuardAi/SoftwarePHP ระบบทำหน้าที่เป็นเกตเวย์รับข้อมูลพิกัดดาวเทียม (GPS), ข้อมูลการก้าวเดิน (Pedometer), และสัญญาณเตือนเหตุการณ์การหกล้มจากอุปกรณ์ฮาร์ดแวร์แบบเรียลไทม์ พร้อมทั้งประมวลผล แปลงพิกัดภูมิศาสตร์ และกระจายการแจ้งเตือนไปยังผู้ดูแลผ่านช่องทางสมาร์ตโฟนและแดชบอร์ดควบคุมส่วนกลางทันทีที่เกิดเหตุ

3.7.1 ออกแบบสถาปัตยกรรมระบบเว็บแอปพลิเคชันติดตามผู้ใช้งาน

สถาปัตยกรรมของระบบ SoftwarePHP ได้รับการออกแบบตามรูปแบบสถาปัตยกรรมซอฟต์แวร์ 3 ระดับ ร่วมกับสถาปัตยกรรมขับเคลื่อนด้วยเหตุการณ์ ดังแสดงในผังระบบ



รูปที่ 3.7.1 ผังสถาปัตยกรรมของระบบ SoftwarePHP

3.7.1.1 รายละเอียดองค์ประกอบในแต่ละระดับ

1) ระดับการนำเสนอ (Presentation Tier / Frontend)

• พัฒนาด้วย **HTML5, CSS3 (Modern Glassmorphism UI), และ JavaScript (Vanilla ES6+)** โดยไม่พึ่งพาเฟรมเวิร์กขนาดใหญ่ เพื่อให้ระบบประมวลผลได้อย่างรวดเร็วและใช้หน่วยความจำต่ำ

• รองรับการแสดงผลแบบตอบสนอง ทั้งบนคอมพิวเตอร์ตั้งโต๊ะของศูนย์ดูแล และหน้าจอมือถือ/แท็บเล็ตของผู้ดูแล

• ผสานไลบรารี **Leaflet.js** ร่วมกับแผนที่ดาวเทียมและผังถนนของ **OpenStreetMap** สำหรับแสดงหมุดพิกัดสดของผู้สวมใส่ เส้นทางการเคลื่อนที่ย้อนหลัง และรัศมีความแม่นยำของสัญญาณ GPS (HDOP)

• ใช้ไลบรารี **Chart.js** ในการพล็อตกราฟแท่งและกราฟเส้นแสดงสถิติจำนวนก้าวเดินรายชั่วโมง สัดส่วนกิจกรรมในแต่ละวัน และคลื่นสัญญาณความเร่งขณะเกิดเหตุ

2) ระดับตรรกะการประมวลผลและการบริการ (Application & Service Tier)

พัฒนาด้วยภาษา **PHP 8.x** โครงสร้างแบบ RESTful API แบบไร้สถานะ ประกอบด้วยโมดูล API บริการหลักดังนี้

○ api/falls.php เกตเวย์รับข้อมูลเหตุการณ์การหกล้ม, คำนวณความรุนแรง, ถอดรหัสพิกัดเป็นชื่อสถานที่, บันทึกสถานะการเข้าช่วยเหลือ, และสั่งการระบบแจ้งเตือนแบบมัลติแชนแนล

○ api/gps.php และ api/position.php รับข้อมูลพิกัดละติจูด/ลองจิจูด, ความเร็ว, ความสูง, จำนวนดาวเทียม, และข้อมูลการนับก้าวเดิน บันทึกลงฐานข้อมูลแบบอนุกรมเวลา

○ api/stats.php คำนวณค่าชี้วัดสถิติสุขภาพรายวัน รายสัปดาห์และรายเดือน เช่น ระยะทางสะสม, เวลาที่มีกิจกรรม, และอัตราการหกล้ม services/notifications.php โมดูลขับเคลื่อนการแจ้งเตือนฉุกเฉิน ทำงานแบบอะซิงโครนัส เชื่อมต่อกับ **Telegram Bot API** และ **Gmail SMTP Server**

3) ระดับความปลอดภัยและการเข้าถึง (Security & Authentication Layer)

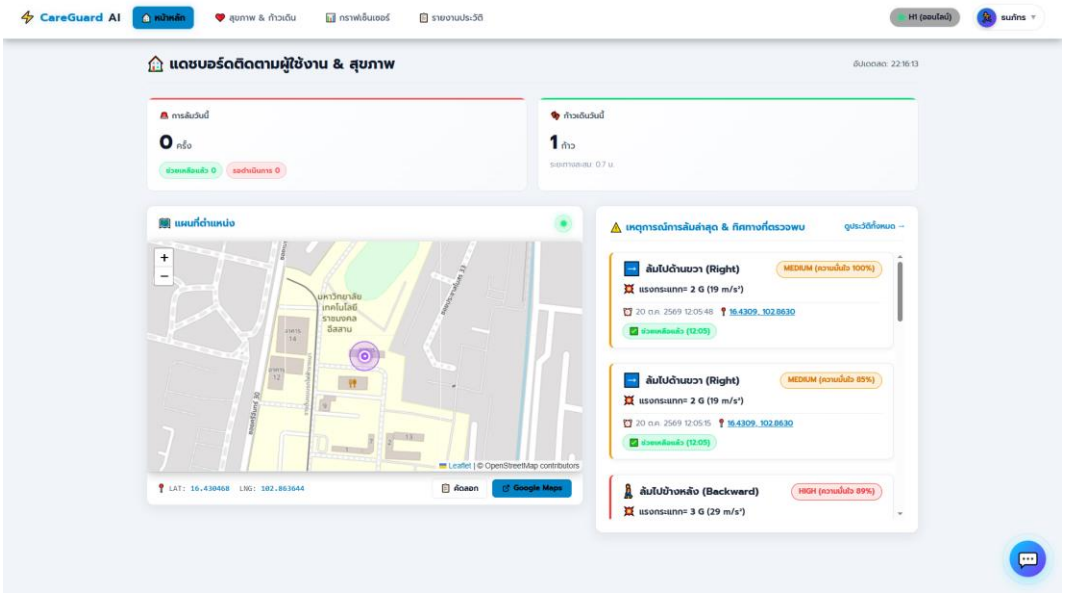
- ป้องกันการเข้าถึงข้อมูลโดยไม่ได้รับอนุญาตผ่านระบบ **API Key Authentication** โดยอุปกรณ์ฮาร์ดแวร์ทุกตัวต้องแนบส่วนหัว X-API-Key ในทุกคำขอ HTTP
- มีระบบจำกัดความถี่ในการส่งคำขอ เพื่อป้องกันการโจมตีแบบ DoS/DDoS
- มีฟังก์ชันคัดกรองข้อมูลนำเข้า เพื่อป้องกันช่องโหว่ SQL Injection และ Cross-Site Scripting (XSS)

4) ระดับการจัดเก็บข้อมูล (Data Storage Tier)

- ใช้ระบบจัดการฐานข้อมูลเชิงสัมพันธ์ **MySQL / MariaDB**
- ใช้กลไกการจัดเก็บแบบ **InnoDB** เพื่อรองรับการทำธุรกรรมแบบ ACID และการล็อกข้อมูลระดับแถว ทำให้สามารถบันทึกพิกัด GPS ความถี่สูงควบคู่กับการอ่านข้อมูลขึ้นแดชบอร์ดได้โดยไม่เกิดปัญหาตารางถูกบล็อก

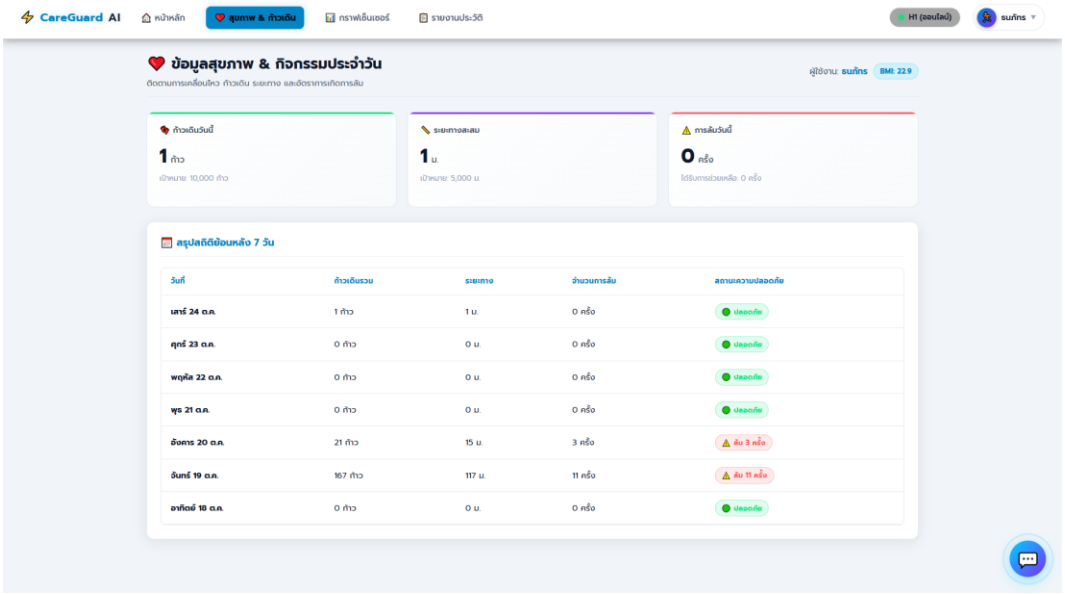
3.7.2 ระบบส่วนติดต่อผู้ใช้และโมดูลการทำงานของเว็บแอปพลิเคชัน

3.7.2.1 หน้าแดชบอร์ดหลักสำหรับติดตามสด



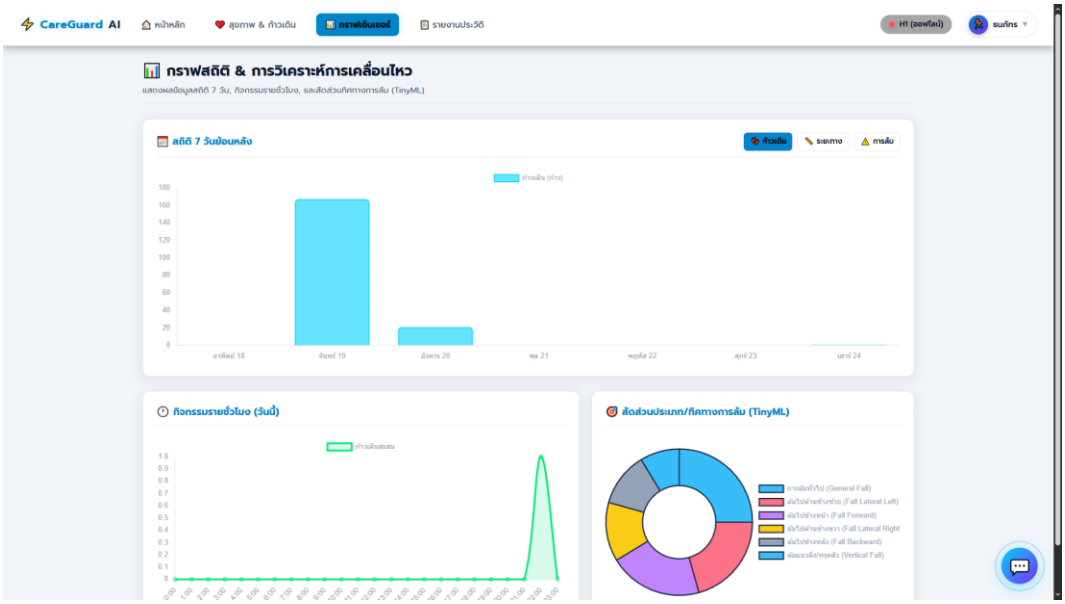
รูปที่ 3.7.2.1 หน้าแดชบอร์ดหลักสำหรับติดตามสด

3.7.2.2 หน้ารายงานสุขภาพและการก้าวเดิน



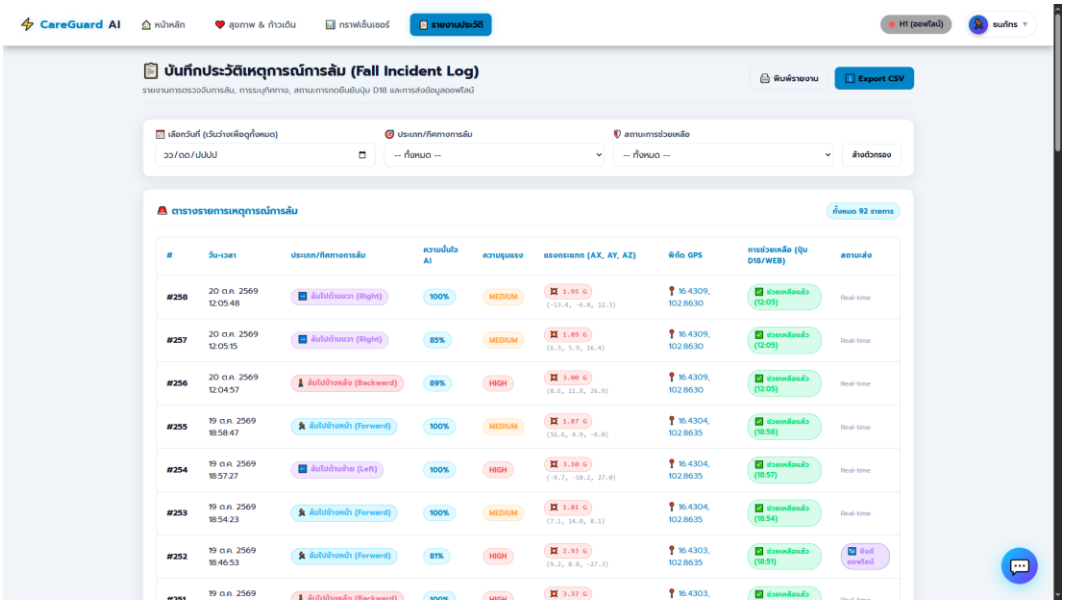
รูปที่ 3.7.2.2 หน้ารายงานสุขภาพและการก้าวเดิน

3.7.2.3 หน้าระบบกราฟวิเคราะห์สัญญาณและการเคลื่อนไหว



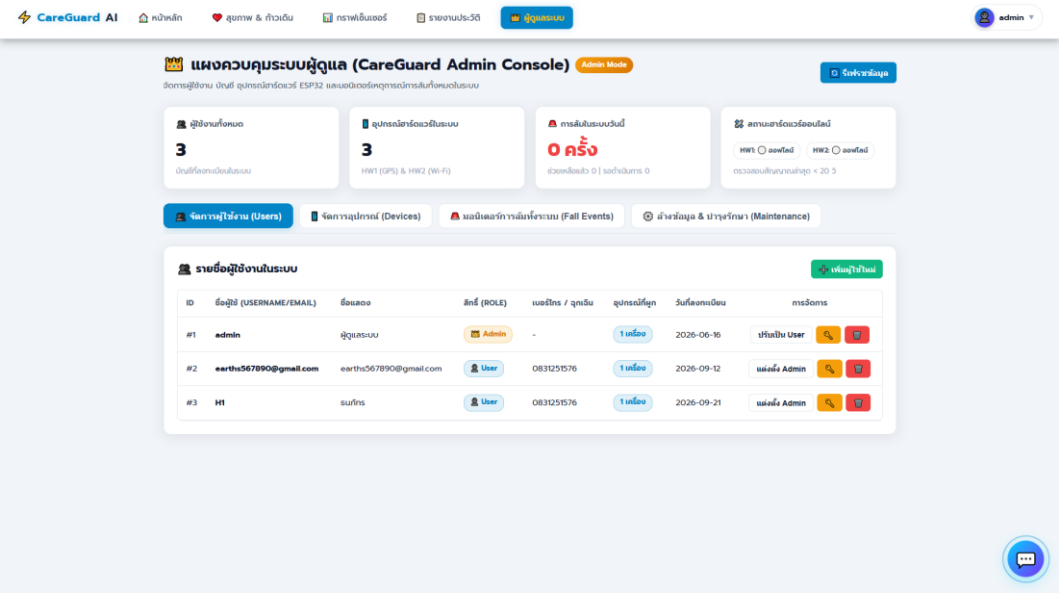
รูปที่ 3.7.2.3 หน้าระบบกราฟวิเคราะห์สัญญาณและการเคลื่อนไหว

3.7.2.4 หน้าระบบรายงานประวัติและการตรวจสอบย้อนหลัง



รูปที่ 3.7.2.4 หน้าระบบรายงานประวัติและการตรวจสอบย้อนหลัง
ที่มา: https://www.youngza.com/IMU/SoftwarePHP/report.html

3.7.2.5 หน้าบริหารจัดการระบบและอุปกรณ์



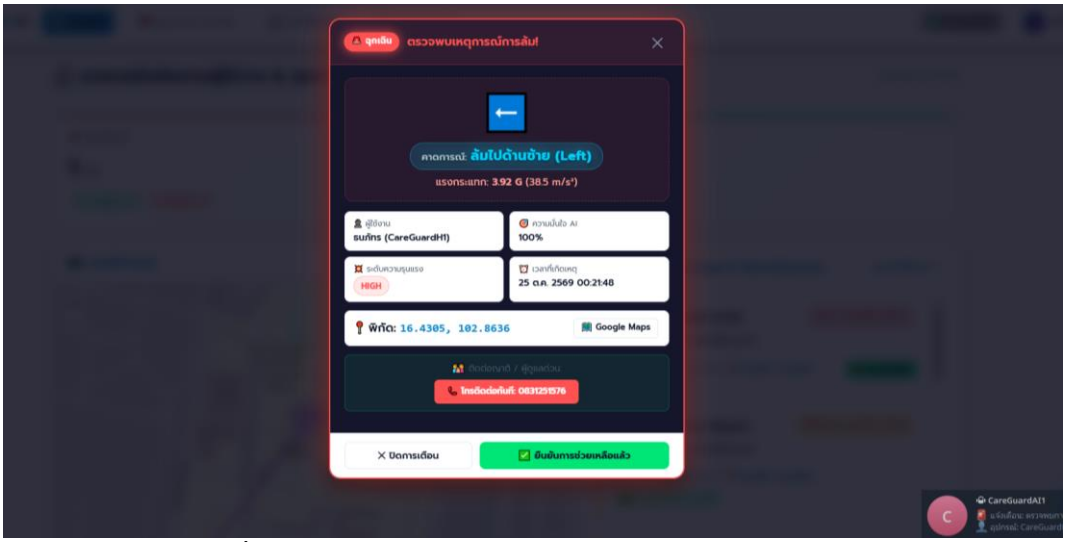
รูปที่ 3.7.2.5 หน้าบริหารจัดการระบบและอุปกรณ์

3.7.3 ระบบแจ้งเตือนฉุกเฉินแบบหลายช่องทาง

เพื่อให้การช่วยเหลือผู้สูงอายุเกิดขึ้นได้ภายในช่วงเวลาวิกฤต แม้ผู้ดูแลจะไม่ได้เปิดหน้าจอคอมพิวเตอร์ทิ้งไว้ ระบบได้เชื่อมต่อบริการส่งข้อความแจ้งเตือนอัตโนมัติผ่าน 3 ช่องทางหลัก

3.7.3.1 การแจ้งเตือนแบบพุชผ่านหน้าเว็บแอพ (SoftwarePHP)

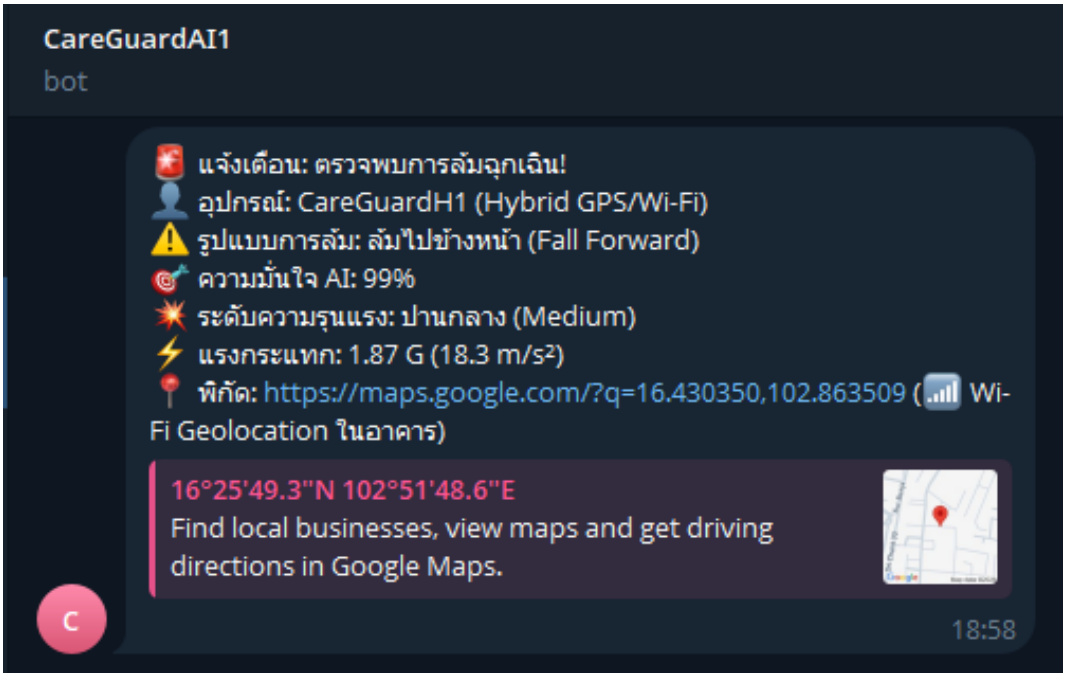
ในการเฝ้าระวังผู้สูงอายุหรือผู้ป่วยที่มีความเสี่ยงสูงต่อการหกล้ม "ระยะเวลาในการรับรู้เหตุการณ์ของผู้ดูแล" ถือเป็นตัวชี้วัดที่มีความสำคัญยิ่งยวดต่อการรอดชีวิตและการลดความพิการ หากระบบอาศัยเพียงการรอให้ผู้ดูแลกดรีเฟรชหน้าเว็บเอง อาจทำให้การเข้าช่วยเหลือล่าช้าจนเกินช่วงเวลานาทีทอง (Golden Period) ผู้จัดทำโครงงานจึงได้ออกแบบและพัฒนากลไกการแจ้งเตือนแบบพุชบนหน้าเว็บแอปพลิเคชัน



รูปที่ 3.7.3.1 ตัวอย่างการแจ้งเตือนแบบพุชผ่านหน้าเว็บแอพ

3.7.3.2 การแจ้งเตือนแบบพุชผ่าน Telegram Bot API

เมื่อเซิร์ฟเวอร์ได้รับข้อมูลการล้ม ฟังก์ชัน sendTelegram() จะทำการแพ็กข้อความแจ้งเตือนความเร่งด่วนสูง ยิงผ่านคำขอ HTTPS POST ไปยังเซิร์ฟเวอร์ของ Telegram ทันที

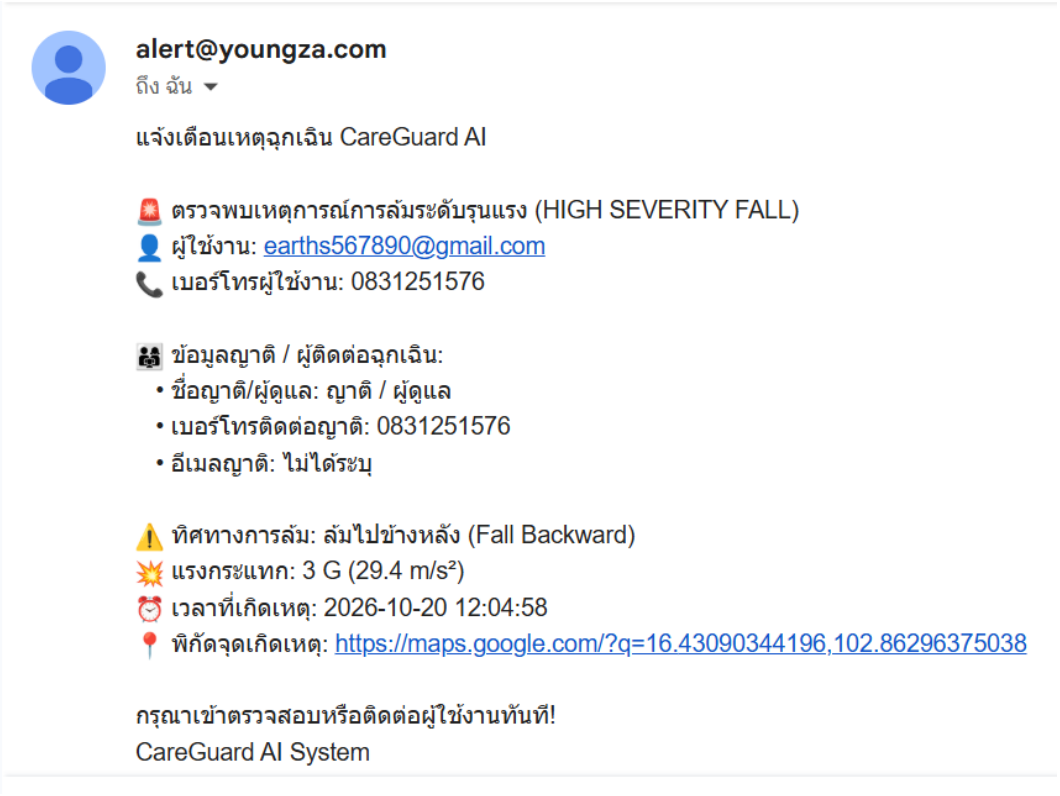


รูปที่ 3.7.3.2 ตัวอย่างการแจ้งเตือนแบบพุชผ่าน Telegram Bot API

3.7.3.3 การแจ้งเตือนผ่านจดหมายอิเล็กทรอนิกส์

• ใช้ไลบรารี **PHPMailer** ส่งอีเมลรายงานรายละเอียดอุบัติเหตุฉุกเฉินแบบ HTML Template อย่างเป็นทางการไปยังอีเมลสำรองของผู้ดูแลหรือแพทย์เวร

• แสดงตารางข้อมูลทางสถิติของเหตุการณ์ เวลาที่เกิดเหตุ และพิกัดสถานที่อย่างละเอียด เพื่อใช้เป็นหลักฐานการติดตามผลการรักษา



รูปที่ 3.7.3.3 ตัวอย่างการแจ้งเตือนผ่านจดหมายอิเล็กทรอนิกส์

3.7.3.4 กลไกการประสานข้อมูลออฟไลน์ (Offline Data Reconciliation)

• หากผู้สูงอายุหกล้มในจุดอับสัญญาณ Wi-Fi อุปกรณ์สวมใส่จะทำการเก็บประวัติการล้ม พิกัด GPS และเวลาลงใน Flash Memory

• เมื่ออุปกรณ์กลับเข้าสู่พื้นที่ที่มีสัญญาณ Wi-Fi อีกครั้ง เฟิร์มแวร์จะส่งข้อมูลย้อนหลังมายังเซิร์ฟเวอร์ เพื่อแจ้งเตือนให้ผู้ดูแลทราบว่ามีเหตุการณ์เกิดขึ้นย้อนหลัง และช่วยป้องกันการสูญหายของข้อมูลสำคัญ

3.8 การเก็บผลการใช้จริง

เพื่อประเมินความถูกต้องแม่นยำและความน่าเชื่อถือของระบบตรวจจับการหกล้มอัจฉริยะ เมื่อนำไปติดตั้งใช้งานจริงบนร่างกายมนุษย์ จึงได้กำหนดระเบียบวิธีการทดสอบระบบแบบครบวงจร โดยให้อาสาสมัครสวมเน็บอุปกรณ์ต้นแบบบริเวณบั้นเอวที่ขอบการเกงหรือเข็มขัด และทำกิจกรรมจำลองเหตุการณ์ในสภาพแวดล้อมจริง จำนวนทั้งสิ้น **150 ครั้ง** แบ่งเป็น

เหตุการณ์การหกล้มจำลอง จำนวน 5 ท่า รวม 50 ครั้ง

ทดสอบบนเบาะรองรับแรงกระแทกความหนา 20 ซม. เพื่อความปลอดภัยของอาสาสมัคร

- ล้มไปข้างหน้า จำนวน 10 ครั้ง
- ล้มไปข้างหลัง จำนวน 10 ครั้ง
- ล้มตะแคงซ้าย จำนวน 10 ครั้ง

- ล้มตะแคงขวา จำนวน 10 ครั้ง
- ล้มแนวดิ่ง / ทรุดตัวลงกับพื้น จำนวน 10 ครั้ง

กิจกรรมในชีวิตประจำวันทั่วไป จำนวน 6 ท่า รวม 60 ครั้ง

ทดสอบรูปแบบการเคลื่อนไหวพื้นฐานในชีวิตประจำวันของผู้สูงอายุ

- เดินปกติบนทางราบ จำนวน 10 ครั้ง
- วิ่งเร็วและวิ่งเหยาะ จำนวน 10 ครั้ง
- เดินขึ้นและลงบันได จำนวน 10 ครั้ง
- นั่งเก้าอี้และลุกขึ้นยืนปกติ จำนวน 10 ครั้ง
- นั่งลงบนพื้นราบและลุกขึ้นยืนเร็ว (Floor Sitting and Rapid Standing) จำนวน 10 ครั้ง
- ก้มเก็บของที่พื้นและผูกเชือกรองเท้า (Bending to Floor / Tying Shoes) จำนวน 10 ครั้ง

กิจกรรมนอกเหนือระบบ/การเคลื่อนไหวความเร่งสูง จำนวน 4 ท่า รวม 40 ครั้ง

เพื่อทดสอบขีดความสามารถและความทนทานของโมเดล TinyML ต่อการเกิดสัญญาณเตือนลวง ในสภาวะที่มีแรงกระแทกหรือแรงเฉื่อยรุนแรงใกล้เคียงการล้ม:

- กระโดดบนพื้นราบ (Jumping on Flat Surface) จำนวน 10 ครั้ง
- ก้าวเท้าสะดุดแต่เสียหลักแล้วทรงตัวได้ จำนวน 10 ครั้ง
- ทิ้งตัวลงนั่งบนเก้าอี้ไม้แข็งอย่างแรงและเร็ว จำนวน 10 ครั้ง
- ทิ้งตัวลงนอนบนเตียงและลุกจากเตียง จำนวน 10 ครั้ง

ตารางที่ 3.8 ตารางบันทึกผลการทดสอบการทำงานของระบบ ในสถานการณ์จริง

ลำดับ	ท่าทดสอบ	จำนวนครั้งที่ทดสอบ	แจ้งเตือนล้ม	แจ้งเตือนแต่ไม่ล้ม	ไม่แจ้งเตือน	อัตราความถูกต้อง (%)
หมวดที่ 1	**เหตุการณ์การหกล้มจำลอง (Falls)**					
1.1	ล้มไปข้างหน้า	10	**10**	0	0	100%
1.2	ล้มไปข้างหลัง	10	**10**	0	0	100%
1.3	ล้มตะแคงซ้าย	10	**10**	0	0	100%

ตาราง 3.8 ตารางบันทึกผลการทดสอบการทำงานของระบบ ในสถานการณ์จริง (ต่อ)

ลำดับ	ท่าทดสอบ	จำนวนครั้งที่ทดสอบ	แจ้งเตือนล้ม	แจ้งเตือนแต่ไม่ล้ม	ไม่แจ้งเตือน	อัตราความถูกต้อง (%)
1.4	ล้มตะแคงขวา	10	**10**	0	0	100%
1.5	ล้มแนวดิ่ง / ทรุดตัวลงกับพื้น	10	**10**	0	0	100%

รวมหมวด 1	**เหตุการณ์การหกล้มจริง**	**50**	**50**	**0**	0	**100%**
หมวดที่ 2	**กิจกรรมในชีวิตประจำวันทั่วไป**					
2.1	เดินปกติบนทางราบ	10	0	**0**	10	100.0%
2.2	วิ่งเร็วและวิ่งเหยาะ	10	0	**6**	4	100%
2.3	เดินขึ้นและลงบันได	10	0	0	10	100%
2.4	นั่งเก้าอี้และลุกขึ้นยืนปกติ	10	0	0	10	100%
2.5	นั่งลงบนพื้นราบและลุกขึ้นยืนเร็ว	10	0	5	5	100%
2.6	ก้มเก็บของที่พื้น	10	0	0	10	100%
รวมหมวด 2	**กิจกรรมประจำวันทั่วไป**	**60**	**0**	**11**	**49**	**81.67%**
หมวดที่ 3	**กิจกรรมนอกระบบ / แรงกระแทกสูง**					
3.1	กระโดดบนพื้นราบ	10	0	**10**	0	0%
3.2	ก้าวเท้าสะดุด	10	0	**0**	10	100%
3.3	ทิ้งตัวนั่งบนเก้าอี้ไม้แข็งแรงและเร็ว	10	0	**1**0	0	0%
3.4	ทิ้งตัวนอนบนเตียงและลุกจากเตียง	10	0	**10**	0	0%
รวมหมวด 3	**กิจกรรมนอกระบบ / แรงกระแทกสูง**	**40**	**0**	**30**	10	**25%**
รวมทั้งหมด	**ครบทั้ง 15 ท่าทาง**	**150**	49	30	71	**79.33%**